

GLM-5: from Vibe Coding to Agentic Engineering

GLM-5 Team

Zhipu AI & Tsinghua University

(For the complete list of authors, please refer to the Contribution section)

Abstract

我们提出 GLM-5，这是一款新一代基础模型，旨在将“vibe coding”范式推进到“agentic engineering”。在前代模型 Agentic、Reasoning 与 Coding (ARC) 能力的基础上，GLM-5 采用 DSA，在保持长上下文保真度的同时显著降低训练与推理成本。为进一步提升模型对齐与自主性，我们实现了新的异步强化学习基础设施，通过将生成与训练解耦，大幅提升后训练效率。此外，我们提出了新型异步 Agent RL 算法，进一步提升 RL 质量，使模型能够更有效地从复杂、长时程交互中学习。凭借这些创新，GLM-5 在主要公开基准上达到最先进水平。更关键的是，GLM-5 在真实编码任务中展现出前所未有的能力，在处理端到端软件工程挑战时超越了此前基线。代码、模型及更多信息见 <https://github.com/zai-org/GLM-5>。

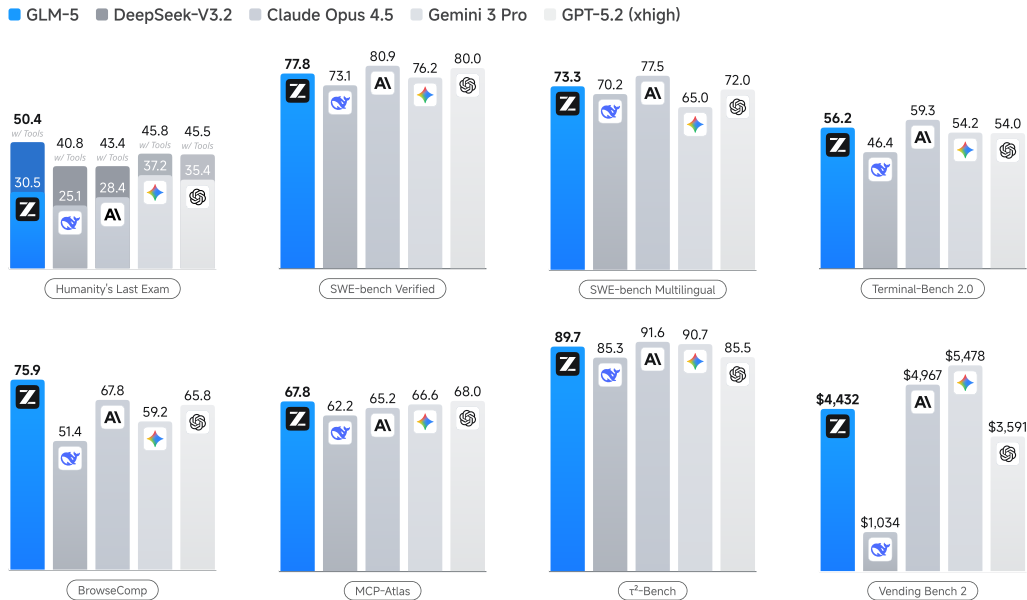


Figure 1: GLM-5、DeepSeek-V3.2、Claude Opus 4.5、Gemini 3 Pro 和 GPT-5.2 (xhigh) 在 8 个智能体、推理与编码基准上的结果：Humanity’s Last Exam、SWE-bench Verified、SWE-bench Multilingual、Terminal-Bench 2.0、BrowseComp、MCP-Atlas、 τ^2 -Bench、Vending Bench 2。

1 引言

追求通用人工智能（AGI）不仅需要扩大模型参数规模，也需要从根本上重思智能效率与自主改进架构。随着 GLM-4.5 的发布，我们证明了将 Agentic、Reasoning 和 Coding (ARC) 能力统一到单一 Model-of-Experts (MoE) 架构中，可以在多类基准上取得最先进结果。然而，

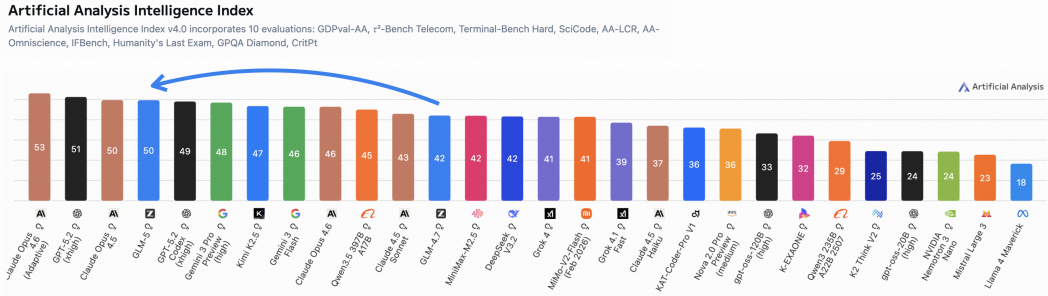


Figure 2: Artificial Analysis Intelligence Index v4.0 包含 10 项评测：GDPval-AA、 τ^2 -Bench Telecom、Terminal-Bench Hard、SciCode、AA-LCR、AA-Omniscience、IFBench、Humanity's Last Exam、GPQA Diamond、CritPt。

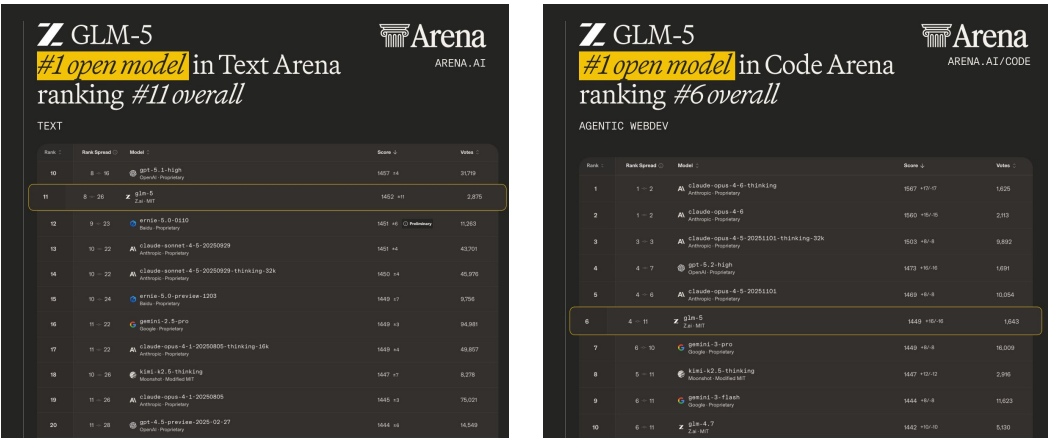


Figure 3: 在 LMArena 上，GLM-5 在 Text Arena 与 Code Arena 均为 #1 开源模型。

随着大语言模型（LLM）从被动知识存储器转向主动问题求解器，计算成本与真实世界适应性（尤其在复杂软件工程中）这两大挑战，已成为主要瓶颈。

我们提出 GLM-5——新一代旗舰模型，旨在突破这些障碍。GLM-5 在性能与效率上都实现了范式跃迁：在 ArtificialAnalysis.ai、LMArena Text、LMArena Code 等主要公开榜单上达到最先进水平。更重要的是，GLM-5 重新定义了真实世界编码标准，展现出处理复杂端到端软件开发任务的前所未有能力，远超 SWE-bench 等传统静态基准所覆盖的范围。

结果。图 1 展示了 GLM-5、GLM-4.7、Claude Opus 4.5、Gemini 3 Pro 与 GPT-5.2 (xhigh) 在 8 个智能体、推理与编码基准上的表现：Humanity's Last Exam [34]、SWE-bench Verified [19]、SWE-bench Multilingual [53]、Terminal-Bench 2.0 [45]、BrowseComp [50]、MCP-Atlas [6]、 τ^2 -Bench [55; 7]、Vending Bench 2 [3]。平均来看，GLM-5 相比上一版本 GLM-4.7 提升约 20%，与 Claude Opus 4.5、GPT-5.2 (xhigh) 大体相当，并优于 Gemini 3 Pro。

GLM-5 在 Intelligence Index v4.0 上达到 50 分，成为新的开源权重榜首（见图 2）；相较 GLM-4.7 的 42 分提升了 8 分，提升主要来自智能体性能与知识/幻觉控制。这也是开源权重模型首次在 Artificial Analysis Intelligence Index v4.0 上达到 50 分。

LMArena 由 UC Berkeley 发起，是一个透明共享的平台，通过人类评判在数百万真实任务上评估并比较前沿 AI 能力，任务包括写作、编码、推理、设计、搜索与创作。大规模人类交互产生了真实世界效用信号，这使其区别于其他静态基准。图 3 显示，GLM-5 再次成为 Text Arena 与 Code Arena 的 #1 开源模型，总体与 Claude-Opus-4.5 和 Gemini-3-pro 处于同一水平。

智能体的长期一致性正变得越来越重要。编码智能体如今可以自主编程数小时，AI 模型可完成任务的长度与广度也将持续提升。我们使用 Vending-Bench 2 与 CC-Bench-V2 两个基准评估 GLM-5 完成长时程任务的能力。Vending-Bench 2 用于衡量 AI 模型在长期经营业务中的表现。模型需要在模拟自动售货机场景中运营一年，并按期末银行账户余额评分。图 4

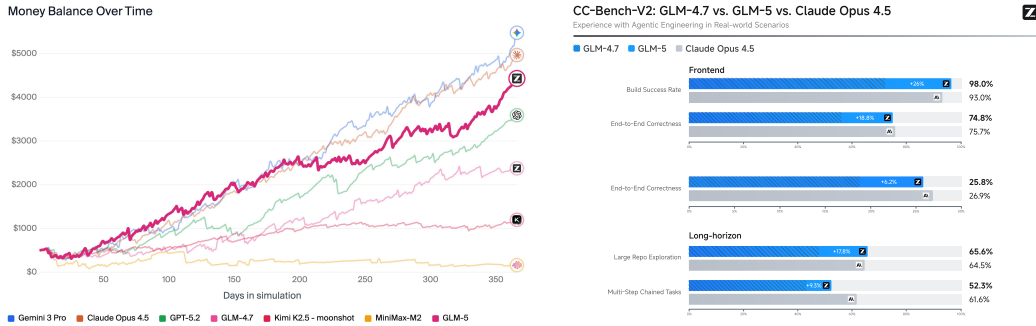


Figure 4: 若干长时程任务结果。左：Vending-Bench 2；右：CC-Bench-V2。

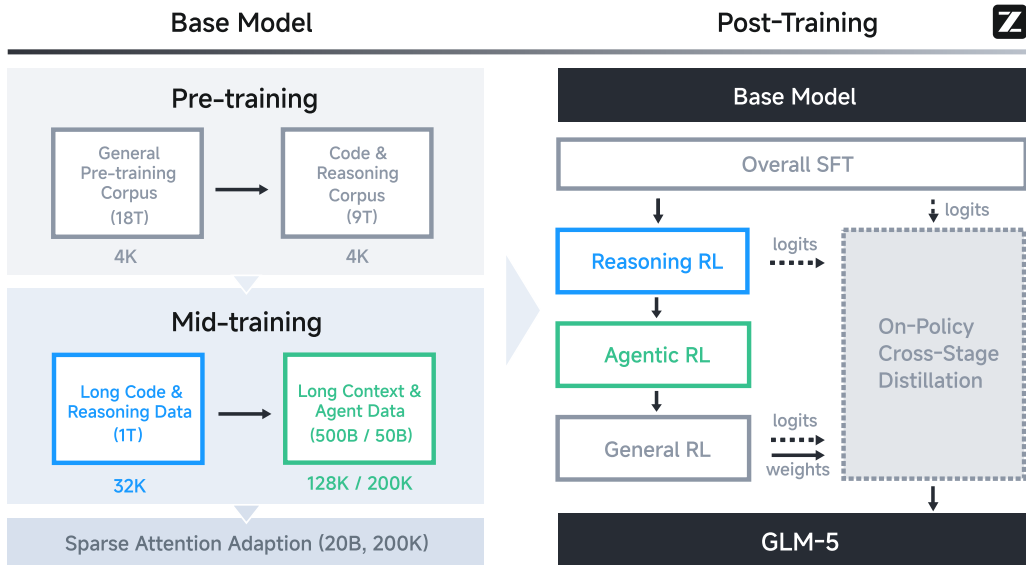


Figure 5: GLM-5 的整体训练流程。

(左) 显示，GLM-5 在所有开源模型中排名 #1，期末账户余额达到 \$4,432。其表现接近 Claude Opus 4.5，体现出较强的长期规划与资源管理能力。图 4（右）进一步展示了我们内部评测套件 CC-Bench-V2 的结果。GLM-5 在前端、后端和长时程任务上均显著优于 GLM-4.7，并缩小了与 Claude Opus 4.5 的差距。

方法。图 5 展示了 GLM-5 的整体训练流程。我们的基座模型训练从一个大规模 27 万亿 token 语料库开始，并在早期优先关注代码与推理。随后我们采用独立的中期训练阶段，将上下文长度从 4K 渐进扩展到 200K，重点使用长上下文智能体数据，以确保复杂 workflows 中的稳定性。在后训练阶段，我们超越了标准 SFT。我们实现了顺序化强化学习流水线——先进行推理 RL，再进行智能体 RL，最后进行通用 RL。关键在于，我们在全过程中使用 On-Policy Cross-Stage Distillation 来防止灾难性遗忘，确保模型在成为强大通才的同时保留锋利的推理能力。总之，GLM-5 的性能跃升主要由以下技术贡献驱动：

首先，我们采用 DSA（DeepSeek Sparse Attention）[9]，这是一项新型架构创新，可显著降低训练与推理成本。尽管 GLM-4.5 已通过标准 MoE 架构提升效率，DSA 仍能依据 token 重要性动态分配注意力资源，在不牺牲长上下文理解和推理深度的前提下大幅降低计算开销。借助 DSA，我们将模型参数规模提升至 744B，并将训练 token 预算扩展至约 28.5T。

其次，我们构建了新的异步强化学习基础设施。基于“slime”框架以及在 GLM-4.5 中初始化的解耦 rollout 引擎，新基础设施进一步将生成与训练解耦，以最大化 GPU 利用率。该系统支持对智能体轨迹进行大规模探索，避免了过去限制迭代速度的同步瓶颈，显著提升了 RL 后训练流水线效率。

第三，我们提出了新型异步 Agent RL 算法，以提升自主决策质量。在 GLM-4.5 中，我们采用迭代式自蒸馏与结果监督来训练智能体。对于 GLM-5，我们开发了异步算法，使模型能够持续地从多样化、长时程交互中学习。这些算法专门针对动态环境中的规划与自我纠错能力进行优化，直接促成了我们在真实世界编码场景中的领先表现。

最后，另一个技术贡献在于：GLM-5 从第一天起就对中国 GPU 生态进行全栈适配。我们已在包括华为昇腾、摩尔线程、海光、寒武纪、昆仑芯、沐曦、燧原在内的 7 个主流国产芯片平台上，完成了从底层 kernel 到上层推理框架的深度优化。

凭借这些进展，GLM-5 不只是一个更强大的模型，更是下一代 AI 智能体更高效、更实用的基础。我们将 GLM-5 开源给社区，以进一步推动高效、智能体化通用智能的前沿发展。

2 预训练

与 GLM-4.5 类似，GLM-5 的基座模型经历两个阶段：用于通用语言与编码能力的预训练，以及用于智能体与长上下文能力的中期训练。我们扩展了 GLM-5 各训练阶段的 token 预算，基座模型总计使用 28.5 万亿 tokens。

2.1 架构

模型规模扩展。GLM-5 将专家数扩展到 256，并将层数降至 80，以降低专家并行通信开销。由此得到 744B 参数模型（40B 激活参数），相较 GLM-4.5（355B 总参数、32B 激活参数）在总规模上翻倍。

Table 1: GQA-8 与 MLA 各变体的评测结果。

Dataset	Hellaswag	MMLU	C-Eval	RACE	BBH	GSM8K	HumanEval
GQA-8	77.3	61.2	60.0	79.6	53.3	47.6	38.5
MLA	77.3	61.5	59.7	77.8	48.9	46.2	33.5
MLA + Muon Split	77.8	62.5	62.1	79.9	51.8	45.0	36.7
MLA-256 + Muon Split	77.4	62.0	59.9	79.6	51.3	47.5	36.6

多潜变量注意力。通过使用降维的 key-value 向量，Multi-latent attention (MLA) [24] 能在效果上匹配 Grouped-Query Attention (GQA)，同时在长上下文序列上带来更优的 GPU 显存节省与更快处理速度。

然而，在使用 Muon 优化器的实验中，我们发现采用 576 维潜变量 KV-cache 的 MLA 无法达到 8 组查询的 GQA（记作 GQA-8，2048 维 KV-cache）性能。为弥合该差距，我们对 GLM-4.5 的 Muon 配方进行改造。在原始配方中，我们对多头查询、键、值的上投影矩阵 W^{UQ}, W^{UK}, W^{UV} 施加矩阵正交化。我们改为将这些矩阵按不同头拆分为更小矩阵，并分别施加矩阵正交化。该方法记作 Muon Split，可使不同注意力头的投影权重以不同尺度更新。Table 1 表明，该方法有效提升 MLA 性能并匹配 GQA-8。实践中我们还发现，在 Muon Split 下，GLM-5 在预训练期间的注意力 logits 尺度保持稳定，无需任何截断策略。

MLA 的另一个缺点是解码时计算成本较高。解码阶段，MLA 需要 576 维点积，高于 GQA 的 128 维计算。虽然 DeepSeek-V3 的注意力头数依据 H800 的 roofline 选取 [60]，但这并不适用于其他硬件。考虑到 MLA 在训练与 prefilling 阶段呈现 Multi-head Attention (MHA) 形态，我们将头维度由 192 提升到 256，并将注意力头数减少 1/3。在保持训练计算量与参数量不变的前提下，这一改动降低了解码计算。该变体在 Table 1 中记作 MLA-256，在 Muon Split 下与 MLA 性能相当。

参数共享的多 token 预测。Multi-token prediction (MTP) [13; 25] 可提升基座模型性能，并可作为 speculative decoding [20] 的草稿模型。然而训练时若要预测后续 n 个 token，需要 n 个 MTP 层。因此，MTP 参数与 kv cache 的显存占用会随投机步数线性增长。DeepSeek-V3 改为只训练单个 MTP 层，并在推理时预测后续 2 个 token，但训练-推理不一致会降低第二个 token 的接受率。为此，我们提出在训练时共享 3

Table 2: DeepSeek-V3.2 与 GLM-5 的接受长度对比。

Model	Accept Length
DeepSeek-V3.2	2.55
GLM-5	2.76

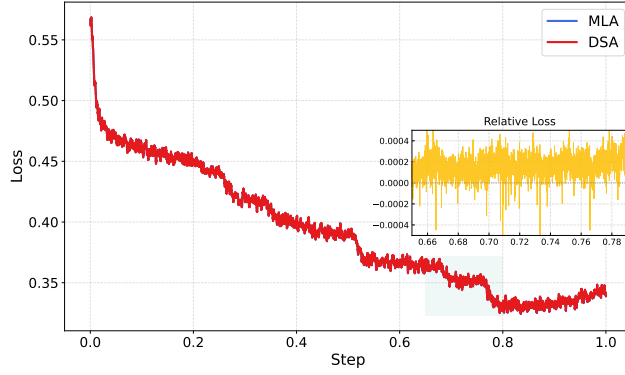


Figure 6: MLA 与 DSA 训练的 SFT loss 曲线对比。结果使用窗口大小为 50 的滑动平均进行平滑。

个 MTP 层参数。这样可在与 DeepSeek-V3 一致的草稿模型内存成本下提升接受率。Table 2 显示，在我们私有提示集上、相同投机步数（4）条件下，GLM-5 的接受长度长于 DeepSeek-V3.2。

2.1.1 使用 DeepSeek Sparse Attention (DSA) 进行继续预训练

Table 3: MLA 与 DSA 基座模型在长上下文基准上的对比。

	MQ-NIAH-128k	MV-NIAH-128k	SQuAD-128k	HotpotQA-128k
MLA	100.0	95.5	79.7	66.3
DSA	100.0	97.0	86.0	63.0

我们在训练中使用 DSA。DSA [9] 的核心思想，是用动态细粒度选择机制替代传统稠密 $O(L^2)$ 注意力——后者在 128K 上下文下成本极高。不同于固定模式（如滑动窗口），DSA 会“查看”内容并决定哪些 token 重要。从研究者视角看，DSA 的一个关键点在于它通过在稠密基座模型上进行继续预训练引入，避免了“天文级”从零训练成本。该迁移遵循“两阶段：稠密预热 + 稀疏训练适配”策略。DeepSeek-V3.2-Exp 在基准性能上保持与其稠密前代一致，证明长上下文中约 90% 的注意力项确实冗余。DSA 在长序列上可将注意力计算降低约 1.5-2 $\times$ ，这对我们构建的推理型智能体非常关键，使其可以在约一半 GPU 成本下处理 128K 上下文。

DSA 训练从中期训练结束时的基座模型启动。预热阶段进行 1000 步，每步使用 14 条长度 202,752 tokens 的序列，最大学习率为 5e-3。随后稀疏适配阶段沿用中期训练的数据与超参，训练 20B tokens。尽管训练预算远小于 DeepSeek-V3.2 (943.7B tokens)，我们发现这已足以将 DSA 模型适配到与原 MLA 模型相当的性能。Table 3 显示，DSA 模型的长上下文性能与 MLA 模型接近。为进一步验证 DSA 训练有效性，我们分别使用相同 SFT 数据微调 DSA 与 MLA 模型，发现两者在训练 loss 与评测基准上持平。

2.1.2 高效注意力变体的消融研究

除 DSA [26] 外，我们基于 GLM-9B¹ 探索了多种替代性高效注意力机制。基线在全部 40 层使用 group query attention，并已在 128K 上下文窗口上完成微调。我们评测以下方法：

- Sliding Window Attention (SWA) Interleave：在网络中统一应用全注意力层与窗口注意力层固定交替模式。
- Gated DeltaNet (GDN) [54]：一种线性注意力变体，以门控线性递推替代二次复杂度的 softmax 注意力计算，将注意力计算成本从序列长度的二次降为线性。

基于这些基线，我们提出两项改进：

¹我们 GLM-4 系列模型之一，见 <https://github.com/zai-org/GLM-4>

- SWA Pattern (基于搜索): 受 PostNAS [15] 启发, 我们提出基于搜索的适配方法, 识别最优层子集进行 SWA 转换, 并在其余层保留全注意力。我们使用 beam search, 在长上下文下游任务上最大化性能。为降低计算成本, 我们仅在 16K 上下文长度进行搜索, 并将所得模式泛化到其他长度。具体而言, beam size 设为 8, 每步优化两层; 对于 40 层的 GLM-9B, 约 10 步收敛。每步在 16K 的 RULER 基准 [17] 上评估候选模式, 并保留前 8 个候选进入下一步。最终模式为 SFSSFFSSSFFFSSFSFFFFFSFSFSFSFSFSFSFS, 其中 S 与 F 分别表示 SWA 与全注意力层。表 4 显示, 该搜索配置显著优于固定交替方式。值得注意的是, 尽管仅在 16K 优化, 该模式在长度泛化上表现稳健, 在所有测试长度上都保持有效。
- SimpleGDN: 一种极简线性化策略, 旨在最大化复用预训练权重, 改进 GDN 在继续训练适配中的表现。我们完全移除 Conv1d 与显式门控模块, 转而将预训练的 Query、Key、Value 投影权重直接映射到线性递推形式。该简化在保留线性注意力效率收益的同时, 消除了额外参数需求。

Table 4: GLM-9B 基线与两种 SWA 变体在不进行任何额外训练条件下的 RULER 结果。两种 SWA 都采用 1:1 的全注意力层与 SWA 层比例, 窗口大小 4096。基于搜索的 SWA 模式仅在 16k 上发现一次, 并统一应用于所有输入长度。

	4K	8K	16K	32K	64K	128K
GLM-9B (Full Attn)	95.19	93.67	92.01	91.09	85.35	75.28
SWA Interleave	94.87	54.02	25.89	12.61	8.32	6.51
SWA Pattern	95.78	92.54	88.92	82.52	70.23	53.95

我们在四个长上下文基准上评测所有方法: RULER [17]、MRCR²、HELMET-ICL [56] 和 RepoQA [27]。结果见表 5。我们在 64K 上下文长度下对每种方法继续训练 190B tokens, 并保持高效注意力层与全注意力层 1:1 比例。对于 GDN 和 SimpleGDN, 我们遵循 Jet-Nemotron [15] 流程。

Table 5: 长上下文基准结果。所有高效注意力变体均从 GLM-9B 全注意力基线继续训练而来。SWA pattern 表示基于搜索的层选择; SWA interleave 表示固定交替模式。 $\Delta@64K$ 与 $\Delta@128K$ 分别表示相对全注意力基线在 64K 和 128K 上下文下的差值。

	RULER (64K/128K)	MRCR (64K/128K)	HELMET-ICL (64K/128K)	RepoQA (64K/128K)
GLM-9B	85.35/75.28	36.53/35.39	77.68/77.36	69.00/65.83
SWA Interleave	65.94/44.93 ($\downarrow 19.41/\downarrow 30.35$)	30.03/28.83 ($\downarrow 6.50/\downarrow 6.56$)	75.96/63.52 ($\downarrow 1.72/\downarrow 13.84$)	50.33/39.33 ($\downarrow 18.67/\downarrow 26.50$)
SWA Pattern	83.72/69.59 ($\downarrow 1.63/\downarrow 5.69$)	35.02/33.58 ($\downarrow 1.51/\downarrow 1.81$)	76.48/74.60 ($\downarrow 1.20/\downarrow 2.76$)	62.33/51.17 ($\downarrow 6.67/\downarrow 14.66$)
GDN	76.76/64.00 ($\downarrow 8.59/\downarrow 11.28$)	31.72/30.22 ($\downarrow 4.81/\downarrow 5.17$)	76.88/74.84 ($\downarrow 0.80/\downarrow 2.52$)	65.50/56.17 ($\downarrow 3.50/\downarrow 9.66$)
SimpleGDN	81.76/67.03 ($\downarrow 3.59/\downarrow 8.25$)	33.03/31.27 ($\downarrow 3.50/\downarrow 4.12$)	79.80/81.84 ($\uparrow 2.12/\uparrow 4.48$)	65.50/58.50 ($\downarrow 3.50/\downarrow 7.33$)

表 5 显示了高效注意力方法间清晰的性能-效率权衡层级。朴素交替的滑动窗口注意力 (SWA) 在长上下文任务上会出现灾难性退化 (例如 RULER@128K 下降 30.35), 而基于搜索的层选择通过在关键层保留全注意力显著缩小了该差距。GDN 等线性注意力变体进一步提升质量, 但需引入额外参数; SimpleGDN 通过最大化复用预训练权重取得了最佳平衡。尽管如此, 这些方法在细粒度检索任务上仍存在固有精度差距——在 RULER@128K 上最多下降 5.69 分, 在 RepoQA@128K 上最多下降 7.33 分——原因在于继续训练适配时高效注意力机制不可避免的信息损失, 即便有一半层保留全注意力。相比之下, DSA 在构造上是无损的: 其 lightning indexer 在不丢弃任何长程依赖的情况下实现 token 级稀疏, 可应用于全部层且不引入质量退化。

为验证这一点, 我们在采用多潜变量注意力的 GLM-4.7-Flash³ 上开展了小规模 DSA 实验。按标准 DSA 配方, 训练分两阶段: (i) *warmup* 阶段仅训练 indexer 1000 步 (batch size 16), 并冻结所有基座模型权重; 随后 (ii) *joint-training* 阶段在 150B tokens 上联合训练模型与 indexer。表 6 给出了 4K 到 128K 上下文长度下的 RULER 结果。仅 *warmup* 变体 (*GLM-4.7-Flash* + *DSA warmup*) 已保留了基线的大部分性能; 性能下降较小且主要集中在最长上下文窗口 (128K: 79.21 $\rightarrow$ 71.35), 短上下文几乎不受影响。完成 150B tokens 联合训练后, *GLM-4.7-Flash* + *DSA* 几乎弥合了全部残余差距: 在 16K (+0.86)、32K (+0.49) 和 64K (+1.72) 上超过基线, 而在 128K 仅落后 0.35 分。

²<https://huggingface.co/datasets/openai/mrcr>

³<https://huggingface.co/zai-org/GLM-4.7-Flash>

Table 6: GLM-4.7-Flash 使用 DSA 的 RULER 结果。仅 warmup 变体只训练 indexer，并保持基座模型冻结；完整 DSA 变体对二者联合训练 150B tokens。

	4K	8K	16K	32K	64K	128K
GLM-4.7-Flash	97.44	96.72	95.83	92.96	85.34	79.21
GLM-4.7-Flash + DSA warmup	97.51	96.54	95.40	90.09	84.05	71.35
GLM-4.7-Flash + DSA	96.77	96.25	96.69	93.45	87.06	78.86

2.2 预训练数据

Web。 在 GLM-4.5 数据流水线基础上，我们优化了海量网页数据的筛选标准。我们引入了另一个基于句向量的 DCLM [21] 分类器，用于识别并聚合标准分类器之外的高质量数据。为应对长尾知识问题，我们使用世界知识分类器（基于 Wikipedia 条目与 LLM 标注数据优化）从中低质量数据中提炼有价值信息。

代码。 我们扩展了代码预训练语料，纳入主要代码托管平台更新快照和更大规模含代码网页，模糊去重后的唯一 token 增长 28%。为提升语料完整性并降低噪声，我们修复了 Software Heritage 代码文件中的元数据对齐问题，并采用更准确的语言分类流程。我们延续 GLM-4.5 的质量感知采样策略，用于源码与代码相关网页文档。此外，我们还为更广泛的低资源编程语言（如 Scala、Swift、Lua 等）训练专用分类器，提升这些语言的采样质量。

数学 & 科学。 我们从网页、书籍与论文收集高质量数学与科学数据，以进一步增强推理能力。具体而言，我们改进了网页内容抽取流水线，以及书籍和论文的 PDF 解析机制，从而提升数据质量。我们使用大语言模型对候选文档打分，仅保留最具教育价值的内容。针对长上下文文档，我们开发了分块聚合打分算法以提升评分准确性。过滤流程严格避免使用合成、AI 生成或模板化数据。

2.3 中期训练

在 GLM-4.5 引入的中期训练框架基础上，我们在 GLM-5 中进一步扩大训练规模与最大上下文长度，以持续增强模型的推理、长上下文与智能体能力。

扩展上下文与训练规模。 我们分三阶段渐进扩展上下文窗口：32K（1T tokens）、128K（500B tokens）和 200K（50B tokens）。相较 GLM-4.5 的 128K 上限，新增的 200K 阶段显著提升了模型处理超长文档和复杂多文件代码库的能力。在后续阶段，我们相应上采样长文档与合成智能体轨迹。

软件工程数据。 我们保留了将仓库级代码文件、commit diff、GitHub issue、pull request 及相关源码拼接为统一训练序列的范式。在 GLM-5 中，我们放宽仓库级过滤标准以扩大可用仓库池，得到约 1000 万组 issue-PR 对；同时加强 issue 级质量过滤以降低噪声。我们还为每组 issue-PR 检索更大规模相关文件，获得更丰富的开发上下文与更广的真实软件工程场景覆盖。过滤后，issue-PR 部分数据约含 160B 唯一 tokens。

长上下文数据。 我们的长上下文训练集由自然数据与合成数据构成。自然数据来自书籍、学术论文和通用预训练语料中的文档，并通过多阶段过滤（PPL、去重、长度）与知识密集领域上采样进行构建。受 NextLong[11] 与 EntropyLong[18] 启发，在合成数据构建中我们采用多种技术构造长程依赖。我们通过交错打包将高相似文本聚合为序列，以缓解 lost-in-the-middle 现象并提升多类长上下文任务性能。在 200K 阶段，我们额外引入少量类似 MRCR 的数据，并设计多个变体扩展 OpenAI 原始范式，以增强超长多轮对话中的召回。经验上我们发现，逐步增加数据多样性持续提升模型长上下文性能；尤其是在初始 128K 阶段之上追加 200K 中期训练后，模型即便在 128K 窗口内的表现也进一步增强。

2.4 训练基础设施

2.4.1 内存效率

灵活的 MTP 放置。 在交错流水并行 [31] 下，模型组件可灵活分配到各 stage。MTP 模块横跨 embedding、transformer 与 output 组件，其内存占用显著高于其他模块，导致 stage 级不

平衡。我们将 MTP 输出层与主输出层共置于最后一段以共享参数，同时将其 embedding 与 transformer 组件放在前一段。这样可降低最后一段内存压力并改善流水各 rank 的均衡性。

Pipeline ZeRO2 梯度分片。每个 pipeline rank 包含多个 stage [31]，朴素实现下每个 stage 都需完整梯度缓冲区用于累积与优化器更新。受 ZeRO2 [38] 启发，我们在数据并行维度对梯度进行分片，使每个 stage 仅存储完整梯度的 $1/dp$ 。另进一步，仅为两个 stage 保留完整累积缓冲，并通过双缓冲复用。当一个 stage 缓冲在连续 microbatch 上累积梯度时，前一 stage 缓冲的梯度同步并行进行。这将持久梯度内存降低为“每 stage 分片缓冲 + 两个完整滚动累积缓冲”，在实践中不增加额外同步开销。

Muon 分布式优化器的零冗余通信。朴素 Muon 实现会在每个数据并行 rank 上 all-gather 全量模型参数，造成瞬时内存峰值和冗余通信。我们将 all-gather 限定为各 rank 持有的参数分片，并将本地计算与分片通信重叠。该方案消除了冗余通信，显著降低了优化器相关峰值内存开销。

流水激活值卸载。在流水 warmup 期间，前向执行先于反向，导致中间激活值生存期拉长。我们在前向后将激活值卸载到主机内存，并在反向前重载 [58]。卸载按层粒度进行，进一步降低峰值内存。结合细粒度重计算后，基本无需将激活值常驻 GPU 显存。卸载与重载被调度为与计算重叠，同时避免与点对点通信及 MoE token 路由（dispatch 与 combination）争用。该方案在近零开销下显著降低激活值内存占用。

序列分块输出投影以降低峰值内存。输出投影与交叉熵损失在反向传播激活保存和损失计算精度提升时会产生瞬时内存开销。为降低该开销，我们将输入序列切分为更小块，对每块独立计算投影与损失，完成该块前后向并释放激活后再处理下一块。块数增加时，峰值内存随之下降。采用合适块数可在保持与不分块执行相近性能的同时，缓解输出层内存压力。

2.4.2 并行效率

高效延迟权重梯度计算。为减少流水气泡，我们将关键路径上的部分权重梯度计算延后 [37]。结合优化后的存储与通信重叠，细粒度延迟可在保持内存开销可控的同时提升吞吐。

高效长序列训练。更长序列会加剧数据并行与流水并行组间的负载不均。我们通过负载感知的序列重排、注意力计算动态重分配，以及将数据并行 rank 灵活划分为不同大小的上下文并行组来解决该问题 [12; 47]。我们还采用分层 all-to-all，对 QKV 张量的节点内与节点间通信进行重叠以降低时延。

2.4.3 INT4 量化感知训练

为在低精度下获得更好精度，我们在 SFT 阶段应用 INT4 QAT。此外，为进一步降低训练时间开销，我们开发了同时适用于训练与离线权重量化的量化 kernel，确保训练与推理间行为按位一致。

3 后训练

GLM-5 的后训练阶段旨在将基座模型转化为具备强大推理、编码与智能体能力的高性能助手。如图 5 所示，我们的流程采用渐进式对齐策略：先进行多任务监督微调（SFT）以引入复杂的交错思考模式，随后进行面向推理与智能体任务的专项强化学习（RL）阶段，最后以面向人类风格对齐的通用 RL 收尾。通过将 on-policy 跨阶段蒸馏作为最终精炼步骤，GLM-5 在充分吸收各阶段收益的同时，有效缓解了能力回退。

3.1 监督微调

与 GLM-4.5 相比，GLM-5 在 SFT 阶段显著扩大了 *Agent* 与 *Coding* 数据规模。GLM-5 的 SFT 语料覆盖三大类别：

- 通用对话（General Chat）：问答、写作、角色扮演、翻译、多轮对话以及长上下文交互；
- 推理（Reasoning）：数学、编程与科学推理；
- 编码 & 智能体（Coding & Agent）：前后端工程代码、工具调用、编码智能体、搜索智能体与通用智能体。

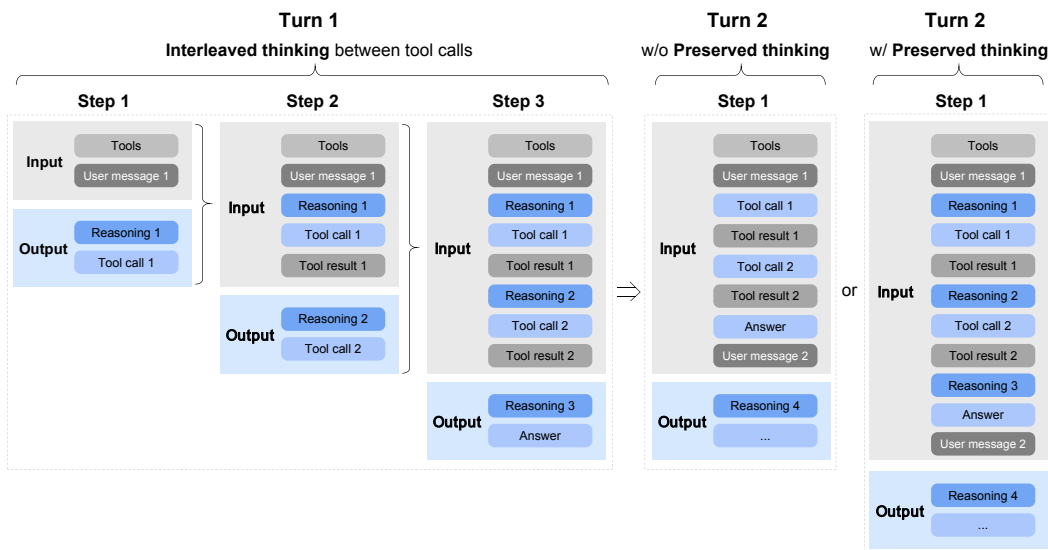


Figure 7: 交错思考（Interleaved Thinking）与保留思考（Preserved Thinking）示意图。

此外，GLM-5 在 SFT 期间将最大上下文长度扩展至 202,752 tokens。结合更新后的聊天模板，模型支持三种不同思考特性（见图 7），包括：

- 交错思考（**Interleaved Thinking**）：模型在每次回复与每次工具调用前进行思考，提升指令遵循与生成质量⁴。
- 保留思考（**Preserved Thinking**）：在编码智能体场景中，模型会在多轮对话中自动保留全部思考块，复用已有推理而非从头重推，减少信息丢失与不一致，特别适用于长时程复杂任务⁵。
- 轮级思考（**Turn-level Thinking**）：模型支持会话内按轮控制推理；对轻量请求可关闭思考以降低延迟/成本，对复杂任务可开启思考以提升准确性与稳定性。

通过在动作间思考并跨轮保持一致性，GLM-5 在复杂任务上实现了更稳定、可控的行为。

对于 通用对话，我们将回复风格优化得比 GLM-4.5 更具逻辑性且更简洁。针对角色扮演任务，我们采集并构建了覆盖多语言和多角色配置的更广泛、更丰富数据集。特别地，我们定义了若干评估维度——包括指令遵循、语言表现力、创造性、逻辑连贯性和长对话一致性——并结合自动与人工过滤进行数据筛选与精炼。

对于 推理任务，我们进一步提升了模型推理深度。具体而言，在逻辑推理方面，我们构建了可验证问题，并通过拒绝采样合成高质量数据；在数学与科学问题方面，我们采用基于难度的过滤流程，仅保留对 GLM-4.7 具有挑战性的题目。

对于 编码与 智能体任务，相较 GLM-4.5，GLM-5 构建了大量执行环境来获取高质量轨迹，并特别强调真实场景与长时程任务。我们进一步使用专家强化学习与拒绝采样改进 SFT 数据。轨迹中的错误片段会被保留，但在 loss 中屏蔽，使模型学习纠错行为而不强化错误动作。

3.2 推理 RL

RL 算法骨干。 我们的 RL 算法基于 GRPO [40]，并引入 IcePop 技术 [61] 以缓解 训练-推理失配，即 RL 优化中推理分布与训练分布不一致的问题。我们显式区分用于梯度更新的 训练策略 π^{train} 与用于轨迹采样的 推理策略 π^{infer} 。相比原始 IcePop 公式，我们移除了 KL 正则

⁴Interleaved thinking 最早由 <https://platform.claude.com/docs/en/build-with-claude/extended-thinking#interleaved-thinking> 引入。

⁵Preserved thinking 也在 Claude Opus 4.5 起采用。见 <https://platform.claude.com/docs/en/build-with-claude/extended-thinking#thinking-block-preservation-in-claude-opus-4-5-and-later>。

项以加速 RL 提升。最终优化损失为：

$$\mathcal{L}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}^{\text{infer}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \text{pop}(\rho_{i,t}, 1/\beta, \beta) \cdot \min\left(r_{i,t} \hat{A}_{i,t}, \text{clip}(r_{i,t}, 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) \hat{A}_{i,t}\right) \right], \quad (1)$$

其中训练-推理失配比定义为

$$\rho_{i,t} = \frac{\pi_{\theta_{\text{old}}}^{\text{train}}(y_{i,t} | x, y_{i,<t})}{\pi_{\theta_{\text{old}}}^{\text{infer}}(y_{i,t} | x, y_{i,<t})}.$$

算子 $\text{pop}(\cdot)$ 会抑制失配比偏离过大的样本：

$$\text{pop}(\rho_{i,t}, 1/\beta, \beta) = \begin{cases} \rho_{i,t}, & 1/\beta \leq \rho_{i,t} \leq \beta, \\ 0, & \text{otherwise.} \end{cases}$$

PPO 风格重要性比与组归一化优势沿用 GRPO 原始定义：

$$r_{i,t} = \frac{\pi_{\theta}^{\text{train}}(y_{i,t} | x, y_{i,<t})}{\pi_{\theta_{\text{old}}}^{\text{train}}(y_{i,t} | x, y_{i,<t})}, \quad \hat{A}_{i,t} = \frac{R_i - \text{mean}(R_1, \dots, R_G)}{\text{std}(R_1, \dots, R_G)}.$$

训练中我们设置超参 $\beta = 2, \epsilon_{\text{low}} = 0.2, \epsilon_{\text{high}} = 0.28$ 。训练完全采用 on-policy，group size 为 32，batch size 为 32。

DSA RL 观察。 我们在基于 DSA 架构的模型上进行了大规模 RL 训练。相较 MLA，DSA 引入额外的 indexer，用于检索最相关的 top-k key-value 条目，并仅在检索子集上稀疏计算注意力。检索得到的 top-k 结果对 RL 稳定性至关重要。这类似于 MoE 模型中使用 routing replay [62] 保留被激活的 top-k 专家，以确保训练-推理一致性。但若直接迁移该策略到 indexer replay，即在每个 token 位置存储 indexer 的 top-k 索引，则明显不可行：indexer 使用的 $k = 2048$ 远大于 MoE 常见的 k ，存储这些索引将产生巨大的存储成本和训练引擎与推理引擎之间的通信开销。

我们发现采用确定性 top-k 算子可有效解决 DSA indexer token 选择中的训练-推理失配。与 SGLang 的 DSA Indexer 中使用的非确定性 CUDA top-k 实现相比，直接使用朴素的 `torch.topk` 虽略慢但具确定性。它能产生更一致输出，并带来显著 RL 增益。相反，其他非确定性 top-k 算子（如 CUDA 或 TileLang 实现）在 RL 仅数步后即出现性能骤降并伴随熵急剧下降。因此，在我们的 RL 阶段中，训练引擎内 DSA Indexer 默认使用 `torch.topk`。此外，我们在 RL 期间默认冻结 indexer 参数，以加速训练并避免 indexer 不稳定学习。

混合域推理 RL。 在推理 RL 阶段，我们在四个领域上进行混合 RL 训练：数学、科学、代码以及工具集成推理（TIR）。对于数学与科学，我们从开源数据集 [10; 30] 与外部标注商共建数据中整理样本。我们还施加难度过滤，聚焦于 GLM-4.7 极少答对或持续失败、但更强教师模型（如 GPT-5.2 xhigh 与 Gemini 3 Pro Preview）可解的问题。代码方面，我们覆盖竞赛编程风格任务与科学编程任务：前者主要来自 Codeforces 及 TACO [23]、SYNTHETIC-2-RL [35] 等代表性数据集；后者由内部题库构建，通过将问题拆解为正确解所需的最小代码实现。TIR 方面，我们复用更具挑战性的数学/科学 RL 数据子集，并与标注商共建明确需借助外部工具作答的 STEM 问题。训练过程中，我们按领域与来源为数据分配特定 judge 模型或评估系统，产出二元结果奖励。我们在四域间维持大致均衡配比，并持续观察到混合 RL 设置下各域稳定且显著提升。

3.3 智能体 RL

为提升 GLM-5 的智能体能力，我们开发了完全异步、完全解耦的 RL 框架，并在编码与搜索智能体任务上优化 GLM-5。朴素同步 RL 在长时程智能体 rollout 中存在严重 GPU 空闲。通过中心化 Multi-Task Rollout Orchestrator 将推理与训练引擎解耦，我们在多样智能体工作负载上实现了高吞吐联合训练。

为在异步 off-policy 条件下维持训练稳定性，我们引入两项关键机制。第一，Token-in-Token-out (TITO) 网关通过保留精确动作级对应，消除了重分词失配。第二，我们采用 Direct Double-sided Importance Sampling，对 rollout log-probabilities 施加 token 级裁剪 ($[1 - \epsilon_\ell, 1 + \epsilon_h]$)，无需追踪历史策略检查点即可有效控制 off-policy 偏差。我们还采用 DP-aware 路由，在大规模 MoE 长上下文推理中最大化 KV-cache 复用以提高速度。为扩展智能体环境规模，我们在三个领域扩展了可验证训练环境：超过 10K 个真实软件工程 (SWE) 任务、终端任务与高难度多跳搜索任务。智能体 RL 细节见后续第 4 节。

3.4 通用 RL

多维优化目标。 我们将通用 RL 的优化目标分解为三个互补维度：基础正确性、情绪智能与任务特定质量。

基础正确性维度是回复质量的基石。它针对会损害模型输出可用性的广泛错误类型，包括指令遵循失败、逻辑不一致、事实错误、知识幻觉与语言不流畅。目标是最小化错误率，使回复达到可用基线。我们将其视为后续优化前提：如果回复包含事实错误或误解用户意图，即使措辞再优雅，也会误导用户。

情绪智能维度在核心正确性之上优化用户体验。它旨在生成更具共情、更有洞察、风格更接近自然人类沟通的回复，让与模型的交互更自然、更有吸引力。

任务特定质量维度面向各类具体任务进行细粒度优化。在基础正确性保障可用性的前提下，它将回复从“仅正确”提升到“高质量”。该维度覆盖写作、文本处理、主客观问答、角色扮演与翻译等多类任务。不同任务域需要不同奖励信号，因此必须采用混合奖励系统。

混合奖励系统。 为监督上述多样目标，我们构建了融合三类互补奖励信号的混合奖励系统：规则奖励函数、结果奖励模型 (ORM) 与生成式奖励模型 (GRM)。三者各有优劣，组合使用是实现稳定、高效、可扩展通用 RL 训练的关键。

规则奖励精确且可解释，但仅适用于可由确定性规则表达的方面。ORM 信号方差低、训练效率高，但更易遭受奖励作弊，即策略利用表面模式而非真正提升核心能力。GRM 利用语言模型产出标量或结构化评估，对此类利用更鲁棒，但往往方差较高。融合三类信号后，我们得到在精确性、效率与鲁棒性之间更平衡的奖励系统，缓解任一单一组件的短板。

Human-in-the-loop 风格对齐。 我们通用 RL 流水线的一项显著特征，是显式纳入高质量人工撰写回复。我们不只依赖模型生成回复，而是将专家人工回复作为风格与质量锚点。动机在于：纯模型自生成优化往往收敛到明显“模型味”的模式——常见问题包括冗长、模板化、缺乏熟练人类写作的细腻度。通过接触人工示例，模型会学习更自然、更符合人类偏好的回复模式。

3.5 On-Policy 跨阶段蒸馏

在多阶段 RL 流水线中，按顺序优化不同目标会导致此前能力累积退化。为缓解该问题，我们将 on-policy 跨阶段蒸馏作为最终阶段，采用 on-policy 蒸馏算法 [14; 52; 51; 28] 快速恢复早期 SFT 与 RL 阶段（推理 RL 与通用 RL）获得的技能。具体地，前序训练阶段的最终 checkpoint 作为教师模型；训练提示从相应教师的 RL 训练集采样，并按合适比例混合。训练损失可通过将式 1 中优势项替换为下式得到（‘sg’表示 stop gradient 操作，例如 `.detach()`）：

$$\hat{A}_{i,t} = \text{sg} \left[\log \frac{\pi_{\theta_{\text{teacher}}}^{\text{infer}}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta}^{\text{train}}(y_{i,t} \mid x, y_{i,<t})} \right]. \quad (2)$$

目前我们使用推理引擎获取教师 logits。未来计划将推理后端迁移至训练引擎，并在推理中统一采用 MLA 的 Multi-Query Attention (MQA) 模式 ($\pi_{\theta_{\text{teacher}}}^{\text{infer}} \rightarrow \pi_{\theta_{\text{teacher}}}^{\text{train}}$)。训练中，GRPO 的 group size 设为 1 以提升数据吞吐，batch size 设为 1024。之所以可行，是因为该阶段不再需要每个 prompt 的大样本组来估计优势；优势可直接由与教师模型的差距计算得到。

3.6 RL 训练基础设施：slime 框架

我们继续使用 slime 作为 GLM-5 统一后训练基础设施，支持端到端大规模强化学习 (RL)。GLM-5 并未引入新的系统组件，而是充分利用 slime 的能力：(1) 通过自由可定制 rollout 与

服务化执行模型扩展任务覆盖；(2) 通过混合精度训练/rollout 与 MTP、Prefill-Decode (PD) 解耦显著提升吞吐——尤其适用于多轮 RL 工作负载；(3) 通过心跳驱动的 rollout 容错与路由层服务生命周期管理提升鲁棒性。

3.6.1 横向扩展：通过高度可定制 Rollout 实现灵活训练

GLM-5 后训练覆盖多样目标。为在不做任务分叉的前提下支持这种多样性，GLM-5 利用 slime 的高度可定制 rollout 接口与服务化 rollout 执行能力。

高度可定制 Rollout。 slime 提供灵活接口以实现任务特定 rollout 逻辑——包括多轮交互循环、工具调用、环境反馈处理与 verifier 引导分支——且无需修改底层基础设施。GLM-5 利用这一能力，在统一训练栈中支持推理 RL、通用 RL、智能体 RL 与 on-policy 蒸馏等广泛域与训练范式。

基于 HTTP API 的服务化 Rollout。 slime 通过标准 HTTP API 暴露 rollout server 与 inference router，用户可像调用常规推理引擎一样与其服务层交互。这使 rollout 逻辑与训练流程边界解耦：外部智能体框架与环境可直接调用 server/router 端点，而优化后端在短时程单轮训练与长时程多轮轨迹中保持不变。

3.6.2 纵向扩展：RL Rollout 的尾延迟优化

对于 RL rollout，优化目标不是总吞吐，而是端到端时延，其主要由每步最慢（长尾）样本主导。实践中，一个拖尾轨迹即可阻塞同步点（如 batch 完成、buffer 就绪、trainer 更新），直接决定墙钟进度。因此 GLM-5 充分利用 slime 面向时延的服务与调度机制，最小化中位延迟，尤其是尾延迟。

通过多节点推理与 MLA 的 DP-attention 实现无队列服务。 为避免排队延迟，即使在突发流量下 rollout 请求也需被及时处理，这要求充足的 KV-cache 容量。GLM-5 采用多节点推理部署（如 8 节点上的 EP64 与 DP64）来提供足够分布式 KV-cache。引入 DP-attention 的主要目的，是避免在不同 rank 间复制 KV。

通过 FP8 rollout 与 MTP 降低尾延迟。 GLM-5 在 rollout 推理中使用 FP8，以降低逐 token 时延并缩短长轨迹完成时间。此外，GLM-5 利用 slime 对 Multi-Token Prediction (MTP) 的支持，这在 RL rollout 常见的小 batch 解码场景中尤为有效。由于尾延迟常由小 BS 拖尾样本驱动（如罕见超长上下文、复杂多轮推理、工具密集轨迹），MTP 对长尾具有不成比例的收益，可提升最慢样本完成时间并降低 step 级阻塞。

PD 解耦防止多轮 RL 中 prefill-decode 相互干扰。 在多轮设置中，长前缀 prefill 很常见（会话历史、工具轨迹、代码上下文）。在 DP-attention 下，若 prefill 与 decode 混用同一服务资源，会产生严重干扰：重 prefill 可能抢占或打断服务器上的 decode，阻碍其他样本持续推进并显著恶化尾延迟。因此 GLM-5 利用 slime 的 Prefill-Decode (PD) 解耦，将 prefill 与 decode 运行在独立资源上，保证 decode 稳定不中断，使长时程样本可持续推进，从而显著改善多轮智能体 RL 的尾部行为。

3.6.3 Rollout 鲁棒性：心跳驱动容错

在大规模场景中，瞬时故障（如单机崩溃、网络问题或性能退化）不可避免。GLM-5 利用 slime 的心跳驱动容错机制，在此类事件下确保训练连续性：rollout server 周期性发送心跳，由编排层监控；不健康服务器会被主动终止并从 inference router 注销。因而重试请求会自动绕开故障或退化服务器路由到健康节点，避免单机事件中断 rollout，保障端到端 RL 训练不中断。

4 智能体工程

我们描述从 **vibe coding**（人类提示）向 **agentic engineering** 的转变。在 vibe coding 中，由人类提示 AI 模型写代码；在 agentic engineering 中，由 AI 智能体自行编写代码。它们能够规划、实现并迭代。为支撑这类长时程任务，GLM-5 采用完全异步、完全解耦的 RL 框架，通过减少智能体 rollout 期间的空闲时间显著提升 GPU 利用率。为扩展智能体环境，我们开发了环境构建流水线。针对编码任务，我们通过构建超过 10,000 个可验证训练场景，搭建了真实软件工程问题与终端任务环境。针对搜索智能体，我们开发了自动化、可扩展的复杂多步推理数据合成流水线，用于构建智能体训练数据。

4.1 面向智能体任务的异步 RL

为在智能体任务上开展 RL，我们设计了完全异步、完全解耦的 RL 基础设施，可高效处理长时程智能体 rollout，并支持跨多种智能体框架的灵活多任务 RL 训练。

我们采用 group-wise policy optimization 算法进行 RL 训练。对每个问题 x ，我们从旧策略 π_{old} 采样 K 条智能体轨迹 $\{y_1, \dots, y_K\}$ ，并按以下目标优化模型 π_θ ：

$$L(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \left[\frac{1}{K} \sum_{i=1}^K (r(x, y_i) - \bar{r}(x)) \right],$$

其中 $\bar{r}(x) = \frac{1}{K} \sum_{i=1}^K r(x, y_i)$ 是采样响应的平均奖励。需要注意的是，优化仅使用模型生成的 tokens，环境反馈不参与 loss 计算。

4.1.1 面向智能体训练的异步 RL 设计

由于 rollout 过程具有长尾特性，朴素同步 RL 训练在 rollout 阶段会因智能体任务生成严重不均衡而产生大量气泡，导致 GPU 空闲时间显著增加。为提升训练吞吐，我们在智能体 RL 中采用完全异步训练范式，以提高 GPU 利用率和训练效率。具体而言，我们将训练引擎与推理引擎解耦并部署在不同 GPU 设备上。推理引擎持续生成轨迹；一旦生成轨迹数达到预设阈值，即将该批次发送至训练引擎进行模型更新。为减小策略滞后并保持训练近似 on-policy，rollout 引擎使用的模型权重会与训练引擎定期同步。训练引擎每进行 K 次梯度更新后更新模型参数，并将新权重回推至推理引擎。尽管异步可显著提升总体训练效率，但这也意味着不同轨迹可能由不同版本模型生成，从而引入严重 off-policy 问题。由于 rollout 策略变化导致权重更新对应的优化问题发生变化，我们也会在每次推理引擎权重更新后重置优化器。

基于服务的多任务训练设计。为应对多任务 RL 中轨迹生成异构性（不同任务通常依赖不同工具集与任务特定 rollout 逻辑），我们引入了基于服务的 Multi-Task Rollout Orchestrator。该组件旨在通过“中央编排器 + 多个注册任务服务”的架构，确保 slime RL 训练框架与多样下游任务的无缝兼容。具体地，每个任务将自身 rollout 与奖励逻辑实现为独立微服务，并注册到中央编排器进行管理与调度。在 rollout 阶段，中央编排器控制各任务 rollout 比例与生成速度，以实现跨任务均衡数据采集。关键在于，我们将所有智能体任务轨迹统一为 message-list 表示。这既支持复杂智能体框架（如软件工程任务）的联合训练，也支持异构负载的集中后处理与日志记录。该设计将任务特定逻辑与核心训练循环清晰隔离，便于无缝接入多任务 RL 训练。作为 GLM-5 训练基础设施的核心骨架，该编排器支持超过 1k 并发 rollout，并支持任务采样比例自动动态调节及任务进度细粒度监控。

4.1.2 异步训练稳定性优化

Token-in-Token-out 与 Text-in-Text-out。 在 RL rollout 场景中，*token-in-token-out* (TITO) 表示训练流水线消费推理引擎产生的精确分词与解码 token 流，并直接用于构建学习轨迹。相对地，*text-in-text-out* 将 rollout 引擎视为返回最终文本的黑盒；训练器随后通过对文本重新分词（并常常重建边界与截断）来重构轨迹，再计算损失。这个看似细微的选择非常关键：重分词会引入 token 边界、空白/规范化处理、截断或特殊 token 放置等细微不一致，进而破坏动作与奖励/优势的步级对齐——尤其在轨迹被流式输出、截断或跨多 actor 交错时更明显。我们发现 token-in-token-out 对异步 RL 训练至关重要，因为它保留了“采样到的动作”与“被优化的动作”之间精确的动作级对应；同时它允许 actor 立即输出轨迹片段（token IDs + 元数据），无需经历有损文本往返，也无需在 learner 端等待事后重分词。实践中，我们实现了 TITO Gateway，拦截 rollout 任务发出的所有生成请求，并记录每条轨迹的 token IDs 与元数据。该设计将繁琐的 token ID 处理与下游智能体 rollout 逻辑隔离，同时避免 RL 训练中的重分词失配。

Direct double-sided importance sampling 的 token 裁剪。 不同于第 3 节中的同步 RL 训练设定，在异步设定下，rollout 引擎在单条轨迹生成期间可能经历多次更新，这使得精确追踪行为策略概率 $\pi_{\theta_{\text{old}}}$ 的计算成本难以接受。否则我们必须维护大量历史模型检查点 $\{\pi_{\theta_{\text{old}}^{(1)}}, \dots, \pi_{\theta_{\text{old}}^{(N)}}\}$ ，在工程上不可行。

为解决该问题，我们首先采用简化的 token 级重要性采样机制，复用 rollout 期间生成的 log-probability 作为直接行为代理。通过将重要性比定义为 $r_t(\theta) = \frac{\pi_\theta}{\pi_{\text{rollout}}}$ 并去除传统 $\pi_{\theta_{\text{old}}}$ ，我

们消除了单独 old-policy 推理的计算开销。其次，我们采用双侧校准的 token 级掩码策略。不同于标准 PPO 的非对称裁剪，我们将信任域限制为 $[1 - \epsilon_\ell, 1 + \epsilon_h]$ ，其中 ϵ_ℓ 和 ϵ_h 为裁剪超参。落在区间外的 token 会被完全从梯度计算中屏蔽，以防极端策略偏移导致不稳定。该方法与 IcePop 机制 [44] 有相似性，但我们的策略进一步移除了 $\pi_{\theta_{\text{old}}}$ ，因此更简单且训练更稳定。

形式化地，带 token 级裁剪的优化目标写为：

$$L(\theta) = \mathbb{E}_t \left[f(r_t(\theta), \epsilon_\ell, \epsilon_h) \hat{A}_t \log \pi_\theta(a_t | s_t) \right] \quad (3)$$

其中重要性比 $r_t(\theta)$ 计算为：

$$r_t(\theta) = \exp(\log \pi_\theta(a_t | s_t) - \log \pi_{\text{rollout}}(a_t | s_t)) \quad (4)$$

稳定性进一步由校准函数 $f(x; \epsilon_\ell, \epsilon_h)$ 保证：

$$f(x; \epsilon_\ell, \epsilon_h) = \begin{cases} x, & \text{if } 1 - \epsilon_\ell < x < 1 + \epsilon_h \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

在实验中我们发现，复用 rollout log-probability 接受了可控程度的 off-policy 偏差，从而避免历史策略追踪需求，同时提升训练稳定性。

丢弃 off-policy 与噪声样本。 在异步 RL 中，过长轨迹可能高度 off-policy，进而破坏训练稳定性。为过滤这类严重 off-policy 样本，我们记录 rollout 引擎在生成时使用的策略权重版本。具体地，对每个响应记录参与生成的模型版本序列 $(w_0, \dots, w_k)$ ，其中 $w_0 < \dots < w_k$ 。令 w' 为当前策略版本。若最早 rollout 版本过旧，即 $w' - w_0 > \tau$ (τ 为预设阈值)，则丢弃该样本。这样可去除与当前策略偏离过远的轨迹。

此外，编码智能体沙箱本身可能不稳定，可能因与模型无关的原因失败（如环境崩溃）。这类失败会引入噪声训练信号，因为它反映的是环境不稳定而非模型能力。为缓解该问题，我们记录每个样本的失败原因，并排除因环境崩溃导致失败的样本。对于 GRPO 等组采样方法，移除失败样本可能导致组不完整。此时若有效样本数超过组大小一半，则通过重复有效样本补齐该组；否则丢弃整组。该流程可减少伪奖励噪声并提升训练稳定性。

DP-aware 路由加速。 我们提出 DP-aware 路由机制，以在数据并行 (DP) 下保持 KV cache 局部性，从而加速大规模 MoE 推理。在多层智能体负载中，同一 rollout 的连续请求共享相同前缀。为最大化 KV 复用，我们施加 rollout 级亲和性：同一智能体实例的所有请求路由到同一 DP rank。具体地，我们引入有状态路由层，使用一致性哈希将每个 rollout ID 映射到固定 DP rank。该映射跨轮保持稳定，避免跨 rank 缓存未命中。为防止长期失衡，我们在哈希基础上加入轻量动态负载再平衡。该设计无需在 DP rank 间同步 KV，即可避免冗余 prefill 计算。随着 rollout 变长，prefill 成本与增量 token 成正比，而非总上下文长度。最终可降低端到端时延并提升长上下文智能体推理的有效吞吐。

4.2 智能体环境扩展

为支持多样智能体任务的强化学习，我们构建可验证、可执行环境，为代码中心任务与内容生成流程提供可落地反馈。针对智能体编码任务，我们开发两条环境构建流水线：一条基于真实软件工程问题的环境搭建流水线，另一条用于终端智能体环境的合成流水线。除编码外，我们还引入了幻灯片生成环境，在该环境中，智能体在结构化 HTML 上操作，并通过可执行渲染与基于布局的验证获得反馈。

4.2.1 软件工程 (SWE) 环境

在构建可执行环境前，我们先收集了大规模真实 Issue-Pull Request (PR) 对话料，并应用严格的规则过滤与 LLM 过滤，以确保获取真实高质量的问题描述。我们将样本划分为多种任务类型——缺陷修复、功能实现、重构及其他，并纳入必要任务约束，确保模型实现与测试补丁一致。我们基于 RepoLaunch [59] 框架构建了环境搭建流水线，可从真实 SWE 问题规模化构建可执行环境。该流水线会自动分析仓库安装与依赖配置，构建可执行环境并生成测试命令；随后利用 LLM 从测试输出自动生成语言感知日志解析函数，以提取 Fail-to-Pass (F2P) 与 Pass-to-Pass (P2P) 测试用例。借助该流水线，我们在数千个仓库上构建了超过 10k 个可验证环境，覆盖 Python、Java、Go、C、CPP、JavaScript、TypeScript、PHP 与 Ruby 共 9 种编程语言。

4.2.2 终端环境

基于种子数据的合成。为规模化构建可验证终端智能体环境，我们设计了包含三阶段的智能体数据合成流水线：任务草案生成、具体任务实现与迭代任务优化。以来自真实软件工程与终端型计算机使用场景的种子任务为起点，我们利用 LLM 进行头脑风暴，生成大量可验证终端任务草案。随后，construction agent 将草案实例化为 Harbor [42] 格式的具体任务，包括结构化任务描述、Docker 化执行环境及对应测试脚本。之后，refine agent 根据人工制定 rubric 审核并迭代优化生成任务，确保 Docker 镜像可稳定构建、测试用例与任务规范一致，并且环境对潜在利用/捷径具有鲁棒性。整体上，该流水线产出数千个多样且可验证的终端智能体环境，Docker 构建正确率超过 90%。

基于网页语料的合成。我们开发了可扩展自动化流水线，基于网页语料构建 LLM 验证的终端式编码任务，并采用闭环设计：构建智能体同时充当其自身首轮评估器。首先，我们收集大规模代码相关网页语料，并应用数据质量分类器仅保留高质量内容，剔除技术性不足或几乎不含实质代码内容的页面。随后从过滤子集中进一步识别适于构造成终端式任务的页面。接着我们按主题类别与难度分层采样，确保最终任务池分布均衡且多样。其次，我们使用 Harbor 任务构建规范⁶（包含任务 schema、格式要求与示例任务）以及所选网页内容来提示编码智能体。智能体被要求：(i) 基于网页内容合成完整终端任务；(ii) 对自身输出执行 Harbor 校验脚本。若校验失败，智能体会迭代诊断并修订任务直至通过所有自动检查。只有通过该自验证闭环的任务才会进入最终数据集。

4.2.3 搜索任务

对于深度搜索型信息检索任务，我们构建了可产出高难多跳问答对的数据合成流水线。每个问题都需要基于来自多个网页来源的证据进行多步推理。

Web Knowledge Graph (WKG) 构建与问题生成。从早期搜索智能体轨迹出发，我们收集并去重其遇到的全部 URL，保留了超过两百万个跨多领域的高信息网页。LLM 执行语义解析，用于实体识别、噪声过滤与结构化信息抽取。WKG 会持续引入新页面并利用下游验证信号迭代精炼，包括实体对齐、属性规范化、关系合并与语义一致性修正。基于 WKG，我们采样低至中频实体作为种子节点，并扩展其多跳邻域形成完整子图，同时控制扩展规模以减少重叠。随后，借助面向高难多域推理的提示，我们将每个子图转换为隐式编码多实体关系链的问题。

高难问题过滤与验证。我们采用三阶段流程平衡难度与正确性：(1) 移除可被无工具推理模型在 8 次独立尝试中至少一次答对的问题。(2) 过滤掉可被早期智能体在少量步骤内通过基础搜索、浏览与计算解决的问题。(3) 使用验证智能体进行双向校验：收集第 2 阶段搜索轨迹中的候选答案，再分别独立验证候选答案与标注真值的问题-答案一致性，拒绝答案不唯一、证据不一致或标签错误样本。最终得到高质量、高难度且可靠的多跳问答对。

4.2.4 搜索智能体的上下文管理推理

我们发现 BrowseComp [50] 上的表现对 judge prompt 与 judge model 都较敏感，开源 judge 可能引入系统性偏差。为确保一致性与可复现性，我们将所有 judge 相关组件统一为官方 OpenAI 评测提示，并使用专有模型 o3-mini 作为 judge。案例研究表明该配置与人工标注真值最一致，因此我们在全部搜索智能体评测中采用该设置。

既有工作 [26] 引入了上下文管理，其中 *Discard-all* 会通过移除全部工具调用历史来重置上下文。我们进一步观察到，在极长上下文（如超过 100k tokens）下模型准确率会明显下降。受此启发，我们采用简单的 *Keep-recent-k* 策略。当交互历史超过阈值 k 时，会折叠最近 k 轮之前的内容以控制上下文长度。设轨迹为 $(q, r_1, a_1, o_1, r_2, a_2, o_2, \dots, r_n, a_n, o_n)$ ，其中 q 为问题， r_i 为第 i 轮推理， a_i 为动作（我们设计 *search*、*open*、*find*、*python* 四种工具）， o_i 为工具观察。我们仅折叠最近 k 轮之前的观察： $o_i \leftarrow \text{Tool result is omitted to save tokens. } i = 1, \dots, n - k$ 。在实验中，我们设 $k = 5$ ，该设置可稳定提升性能，使 GLM-5 从 55.3%(w/o keep-recent-k) 提升到 62.0%(w/ keep-recent-k)。我们还发现，采用不同 keep recent k 值，或改为在上下文长度达到预设 token 阈值时触发 keep-recent，结果基本一致。

在此基础上，我们将 keep-recent 与 Discard-all 结合，形成混合 *Hierarchical Context Management* 策略。在 keep-recent 推理期间，若总上下文长度超过阈值 T ，我们丢弃全部工具调用历史并在新上下文中重启，同时继续应用 keep-recent 策略。我们通过参数搜索选取 $T = 32k$ 。

⁶<https://harborframework.com/docs/tasks/task-tutorial>

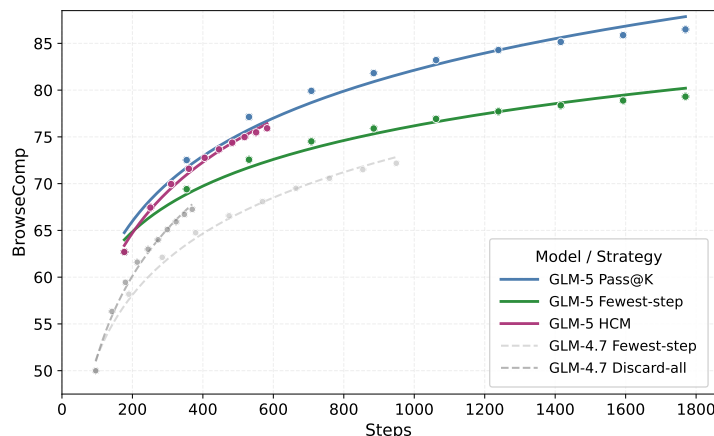


Figure 8: 从 GLM-4.7（灰色基线）到 GLM-5（彩色策略），BrowseComp 在不同上下文管理策略下的准确率。

如图 8 所示，在不同算力预算下，该策略可有效释放上下文空间，使模型执行更多步骤并持续提升性能。与仅使用 Discard-all 相比，结合 keep-recent-k 在所有预算上均有稳定增益，最终分数达 75.9，超过所有采用上下文管理的开源模型。

4.2.5 幻灯片生成

我们采用自我提升流水线，目标是通过强化学习与拒绝采样微调训练专门的幻灯片生成专家，从而系统性提升幻灯片生成性能。我们先用监督微调（SFT）初始化模型的基础幻灯片生成能力，然后在常见审美与结构属性驱动的多层级奖励下执行强化学习。该阶段显著提升了生成质量。随后我们进行拒绝采样微调与掩码微调，使强化学习获得的知识回注训练语料。该流程以协同迭代方式同时提升数据质量与模型能力。

我们提出 多层级奖励构造，将基于 HTML 的幻灯片生成过程中的奖励信号划分为三层：

Level-1: 静态标记属性。该层关注生成 HTML 中的声明式属性，包括位置、间距、颜色、字体、饱和度等样式属性。基于专业设计原则，我们设计了一组规则来约束模型在生成这些声明时的行为。这些规则既保证生成 HTML 的语法可解析性，也将标记级设计空间约束在更利于表现力、结构清晰度、视觉和谐与可读性的子空间。此外，我们引入了幻觉图片与重复图片检测机制，以抑制幻觉或冗余图像。

Level-2: 运行时渲染属性。与静态检查不同，该层评估渲染过程中 DOM 节点的运行时属性，如元素宽高、边界框及其他几何布局指标。通过约束这些属性，我们鼓励生成幻灯片在空间组织上更符合人类审美偏好。我们开发了分布式渲染服务，能够高吞吐执行渲染任务并提取所需运行时属性。训练中我们观察到多种奖励作弊行为，例如对超长内容硬截断或过度操控间距（见图 9）。为缓解这些问题，我们改进了渲染器实现，消除可利用漏洞，确保奖励信号真正激励审美一致布局，而非对几何指标的表面迎合。

Level-3: 视觉感知特征。在运行时渲染约束之外，我们引入对渲染结果的感知层评估。例如，我们检测异常留白模式作为辅助信号，进一步改善整体构图平衡与视觉美感。

训练策略。训练中联合优化上述信号，以提升生成 HTML 的结构有效性、增强布局组织并提升整体视觉审美质量。除奖励设计外，我们还通过动态采样重塑训练分布。具体地，以概率方式丢弃部分结构过于简单样本，使优化聚焦更具挑战性的页面并提升复杂构图场景鲁棒性。我们还采用 token 级 policy gradient loss 稳定优化 [57]。此外，我们引入平衡策略，将同一样本不同 rollout 结果分配到多个训练 batch，以降低优化偏差并提升训练稳定性。

拒绝采样。在拒绝采样阶段，RL 中使用的奖励函数被迁移到数据过滤流水线，以构建高质量训练子集。页面级过滤标准包括代码有效性与可编译性；轨迹级进一步约束工具执行正确性与全局内容多样性，以确保结构一致性。我们采用 Best-of- N 选择策略，即从多个独立生成候选中保留质量最高样本。该机制可有效将分布重加权到更高质量样本，从而提升样本效率与训练稳定性。

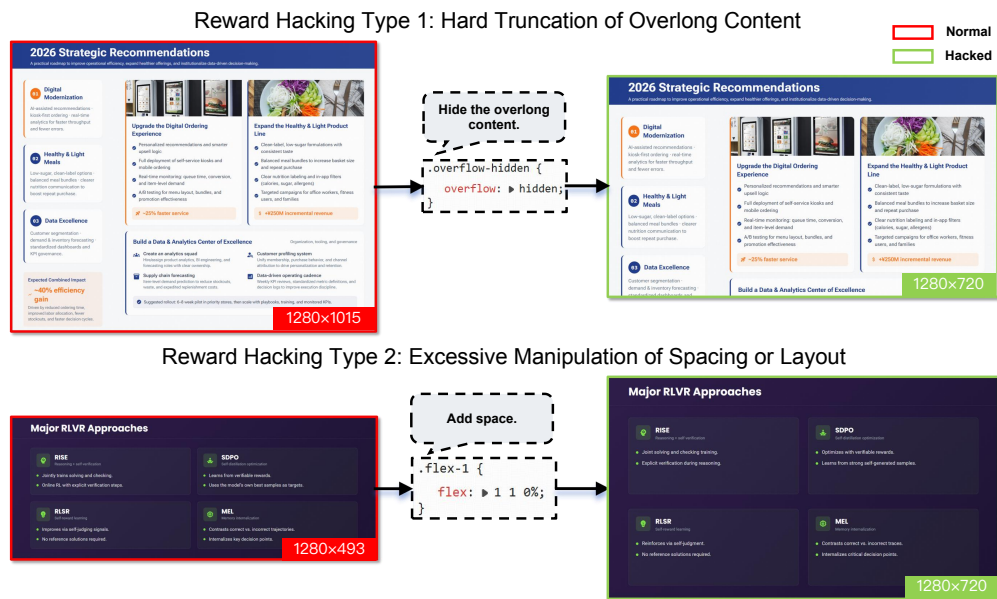


Figure 9: 幻灯片 RL 训练中的奖励作弊示例。我们的运行时渲染可获取真实属性值，使评估对这类作弊行为更稳健。

基于掩码的细化。 尽管拒绝采样可移除大多数低质量输出，但部分轨迹缺陷仅局限于少数页面。若直接丢弃这类样本，会降低有效数据利用率并提高生成成本。为此，我们引入基于掩码的修正机制，自动识别缺陷页面并进行掩码处理，同时保留同一轨迹中的高质量内容。该选择性细化可保留有价值监督信号、提升有效数据效率并减少冗余再生成开销，从而提高整体训练效率。

实证提升。 严格符合 16:9 宽高比的生成页面占比从 40% 提升至 92%，并伴随页面溢出案例显著下降。人工评测进一步显示，相较 GLM-4.5，GLM-5 在内容质量、布局合理性与视觉美感上的胜率分别达到 60%、57.5%、65%，总体胜率达 67.5%。这些结果为所提出多层级奖励设计与自我提升框架的有效性提供了实证支持。

5 GLM-5 对国产芯片基础设施的适配

由于硬件生态异构性高、常常增加高性能部署复杂度，将 GLM-5 适配到多样国产芯片基础设施面临显著挑战。尽管如此，我们已与华为昇腾、摩尔线程、海光、寒武纪、昆仑芯、沐曦、燧原七大主流国产芯片平台紧密协作，成功实现 GLM-5 全栈适配。本节以昇腾 Atlas 系列为案例，围绕三大核心支柱介绍我们的适配方法：极限量化、高性能 kernel 融合与先进推理引擎调度。

混合精度 W4A8 量化。 为将 750B 参数规模的 GLM-5 部署到单台 Atlas 800T A3 机器上，我们实现了复杂的 W4A8 混合精度量化策略。借助 msModelSlim⁷ 工具，我们对不同组件施加不同精度：标准 Attention 与 MLP block 使用 W8A8 (INT8)，而 MoE experts 压缩为 W4A8 (INT4)，在几乎不损失精度的前提下大幅降低内存占用。我们还采用 QuaRot [2] 进行离群值抑制，并使用 Flex_AWQ_SSZ 做缩放校准，以维持低比特部署稳定性。

高性能融合 kernel。 为突破昇腾 NPU 上稀疏注意力计算瓶颈，我们开发了一套定制融合 kernel：Lightning Indexer、Sparse Flash Attention、MLAPO (Multi-head Latent Attention Pre-processing Optimization)。Lightning Indexer 将分数计算、ReLU 与 TopK 操作融合为单一 kernel，使 NPU 能够重叠计算与访存。对于 Sparse Flash Attention kernel，我们针对 GLM-5 的稀疏模式做了专项优化。该 kernel 可并行处理 KV cache 中 TopK token 选择与稀疏注意力

⁷https://www.hiascend.com/document/detail/zh/CANNCommunityEdition/80RC3alpha003/devaids/auxiliarydevtool/modelslim_0001.html

计算。最后，MLAPO 将 13 个小型预处理算子融合为一个“超级算子”，并利用 Vector 与 Cube 单元并行处理以提升端到端效率。

专用推理引擎优化。 我们改造了两款主流推理引擎 vLLM-Ascend 与 SGLang，以最大化硬件利用率：

- **异步调度：**在 vLLM 中，我们实现了将“Device-to-Host”（D2H）采样拷贝与下一步 decode 准备重叠的机制，有效消除调度“气泡”。
- **上下文管理：**RadixCache（前缀共享）与 Prefix Cache（将 KV 存储扩展到系统内存）等特性可高效复用 KV 条目，这对长上下文性能至关重要。
- **并行策略：**我们采用 Attention Data Parallelism（DP）与 MoE Expert Parallelism（EP）的混合策略，并配合 FlashComm，通过拆分 AllReduce 操作将通信时延隐藏在计算之后。
- **Multi-Token Prediction (MTP)：**通过每步推理生成多个 token，我们显著提升了 NPU 计算密度并减少了总序列生成时间。

通过这些硬件层协同优化，GLM-5 在单个国产节点上的性能可比拟国际双 GPU 集群，同时在长序列场景下将部署成本降低 50%。

6 评估

如前所述，GLM-5 标志着从 vibe coding 向 agentic engineering 新时代的过渡。我们首先在智能体、推理与编码（ARC）基准上，将 GLM-5 与前沿模型进行评测。为全面评估 GLM-5 在真实智能体工程场景中的表现，我们提出新的内部评测套件 CC-Bench-V2，覆盖前端、后端与长时程任务。最后，我们在五类常见真实场景中评估 GLM-5 的通用能力。

6.1 ARC 基准评估

我们在表 7 中报告 ARC 基准主要结果，对比 GLM-5 与 GLM-4.7、DeepSeek-V3.2 [26]、Kimi-K2.5 [43]、Claude Opus 4.5 [1]、Gemini 3 Pro [8] 以及 GPT-5.2 (xhigh) [32]。总体而言，GLM-5 相较 GLM-4.7 实现显著提升，并在开源模型中达到最先进性能，缩小了与 Claude Opus 4.5 等闭源模型的差距。评测细节见第 B.2 节。

6.1.1 推理与通用基准评估

在推理与通用基准方面，我们评测了 Humanity’s Last Exam (HLE) [34]、AIME 2026、HMMT 2025、IMO-AnswerBench [29]、GPQA-Diamond [39] 以及 LongBench v2 [5]。对于 HLE，我们仅评测文本子集，并使用 GPT-5.2 (medium) 作为 judge 模型。多数推理任务采用 131,072 的最大生成长度，而 HLE-with-tools 使用 202,752 最大 tokens。

从表 7 可见，GLM-5 在推理任务上与强开源基线 Kimi-K2.5 表现相当。与闭源模型相比，GLM-5 在 HLE (with tools) 上优于 Claude Opus 4.5 和 Gemini 3 Pro。相较前代 GLM-4.7，GLM-5 在 HLE (含/不含工具) 上均有显著提升。在 HMMT Feb./Nov. 2025 上，GLM-5 优于 Claude Opus 4.5 与 Gemini 3 Pro。GLM-5 在长上下文任务上也取得显著进展，在长上下文推理基准 LongBench v2 中取得仅次于 Gemini 3 Pro 的第二高分。

6.1.2 编码基准评估

在编码基准方面，我们在 SWE-bench Verified [19]、SWE-bench Multilingual [53]、Terminal Bench 2.0 [45] 与 CyberGym [48] 上评测 LLM。对 SWE-bench Verified & Multilingual，我们在 OpenHands 框架中使用为 GLM-5 定制的指令提示。对 Terminal-Bench 2.0，我们采用 Terminus-2 与 Claude Code 两个智能体框架，并额外报告修复部分歧义指令后的 verified Terminal-Bench 2.0 表现⁸。CyberGym 在 Claude Code 2.1.18 中评测。

从表 7 可见，GLM-5 在开源 LLM 中于编码基准上达到 SOTA。与闭源 LLM 相比，GLM-5 在 SWE-bench Verified 上优于 Gemini 3 Pro，在 SWE-bench Multilingual 上也优于 Gemini 3 Pro 与 GPT-5.2 (xhigh)。在 Terminal-Bench 2.0 上，GLM-5 与 Claude Opus 4.5 结果相当，并在修复歧义指令后表现更优。为展示编码能力泛化性，我们在两个智能体框架上评测 Terminal

⁸更多信息见 <https://huggingface.co/datasets/zai-org/terminal-bench-2-verified>

Table 7: GLM-5 与开源/闭源模型对比。标记 * 的结果来自 HLE 全集集合。标记 [†] 的结果在经过核验的 Terminal-Bench 2.0 上评估，修复了部分歧义指令。GDPval-AA Elo 记录于 2026 年 2 月 15 日。每个基准最高分以 **粗体标注**，第二高分以下划线标注。

	GLM-5	GLM-4.7	DeepSeek -V3.2	Kimi K2.5	Claude Opus 4.5	Gemini 3 Pro	GPT-5.2 (xhigh)
<i>Reasoning & General</i>							
HLE	30.5	24.8	25.1	31.5	28.4	37.2	<u>35.4</u>
HLE (w/ Tools)	<u>50.4</u>	42.8	40.8	51.8	43.4*	45.8*	45.5*
AIME 2026 I	92.7	<u>92.9</u>	92.7	92.5	93.3	90.6	-
HMMT Feb. 2025	<u>97.9</u>	97.1	92.5	95.4	92.9	97.3	99.4
HMMT Nov. 2025	<u>96.9</u>	93.5	90.2	91.1	91.7	93.0	97.1
IMO-AnswerBench	82.5	82.0	78.3	81.8	78.5	<u>83.3</u>	86.3
GPQA-Diamond	86.0	85.7	82.4	87.6	87.0	<u>91.9</u>	92.4
LongBench v2	<u>64.5</u>	59.1	59.8	61.0	64.4	68.2	59.8
<i>Coding</i>							
SWE-bench Verified	77.8	73.8	73.1	76.8	80.9	76.2	<u>80.0</u>
SWE-bench Multilingual	<u>73.3</u>	66.7	70.2	73.0	77.5	65.0	72.0
Terminal-Bench 2.0 (Terminus-2)	<u>56.2</u> / 60.7 [†]	41.0	39.3	50.8	59.3	54.2	54.0
Terminal-Bench 2.0 (Claude Code)	<u>56.2</u> / 61.1 [†]	32.8	46.4	-	57.9	-	-
CyberGym	<u>43.2</u>	23.5	17.3	41.3	50.6	39.9	-
<i>Agentic</i>							
BrowseComp	62.0	52.0	51.4	<u>60.6</u>	37.0	37.8	-
BrowseComp (w/ Context Manage)	75.9	67.5	67.6	<u>74.9</u>	57.8	59.2	65.8
BrowseComp-ZH	<u>72.7</u>	66.6	65.0	62.3	62.4	66.8	76.1
τ^2 -Bench	89.7	87.4	85.3	80.2	91.6	<u>90.7</u>	85.5
MCP-Atlas (Public Set)	<u>67.8</u>	52.0	62.2	63.8	65.2	66.6	68.0
Tool-Decathlon	39.2	23.8	35.2	27.8	<u>43.5</u>	36.4	46.3
Vending-Bench 2	\$4,432	\$2,377	\$1,034	\$1,198	<u>\$4,967</u>	\$5,478	\$3,591
GDPval-AA Elo	<u>1,409</u>	1,198	1,195	1,288	1,400	1,201	1,462

Bench 2.0, GLM-5 在两者上均表现稳定。在网络安全编码基准 (CyberGym) 上, GLM-5 相比 GLM-4.7 显著提升, 仅次于 Claude Opus 4.5。

6.1.3 智能体能力评估

在智能体基准方面, 我们评测了 GLM-5 与前沿模型在 BrowseComp [50]、BrowseComp-ZH [63]、 τ^2 -Bench [7]、MCP-Atlas [6]、Tool-Decathlon [22]、Vending-Bench 2 [3] 与 GDPval-AA [33] 上的表现。BrowseComp 衡量语言智能体通过网页浏览解决挑战性问题的能力, BrowseComp-ZH 主要面向中文网页。我们在 BrowseComp 中使用 discard-all 作为上下文管理策略, 与 DeepSeek-V3.2 和 Kimi K2.5 一致。 τ^2 -Bench 评估对话智能体在双控制环境中的能力。我们对 Retail 与 Telecom 加入小幅提示调整, 以避免过早用户终止导致失败 (见 B.3)。对 Airline, 我们应用 Claude Opus 4.5 system card [1] 中提出的领域修复以获得更准确结果。MCP-Atlas 是真实工具使用基准, 在给定 Model Context Protocol (MCP) 服务器条件下评估 LLM 在多步工作流中的表现。为公平对比, 我们在 500 任务公开集上重评全部模型, 并将单任务超时从 4 分钟扩展到 10 分钟, 以避免部署条件导致任务失败。MCP-Atlas 使用 Gemini 3 Pro 作为 judge。Tool-Decathlon 同样是工具使用基准, 但面向真实长时程任务。Vending-Bench 2 在模拟环境中衡量 LLM 在长时间跨度商业场景中的智能体能力, 相比前代 Vending-Bench 引入更多真实因素。GDPval 聚焦 AI 智能体在具有经济价值任务上的表现。

从表 7 可见，GLM-5 在智能体基准上相较 GLM-4.7 显著提升。在 BrowseComp（含/不含上下文管理）上，GLM-5 在前沿 LLM 中达到 SOTA。在 BrowseComp-ZH 上，GLM-5 也优于 Claude Opus 4.5 与 Gemini 3 Pro。在三项工具使用智能体任务（ τ^2 -Bench、MCP-Atlas、Tool-Decathlon）上，GLM-5 与 Claude Opus 4.5 表现相当，显示出 GLM-5 的强工具使用能力。GLM-5 在 Vending-Bench 2 上的结果（\$4,432）进一步证明其商业任务长时程能力。在经济场景中，GLM-5 在 GDPval-AA 上优于 Claude Opus 4.5，仅次于 GPT-5.2（xhigh）。

6.2 真实智能体工程体验评估

真实体验比榜单更重要。我们将内部 CC-Bench 升级为 CC-Bench-V2，用于评估模型在真实智能体工程环境中能否正确完成前端、后端与长时程端到端任务。CC-Bench-V2 完全去除人工标注，基于 Claude Code 与其他 agent harness，结合单元测试和 Agent-as-a-Judge 技术实现全自动评估。

前端。我们先构建智能体生成的前端项目，并检查语法、依赖与兼容性错误。随后使用 Agent-as-a-Judge，通过配备 Playwright 与 bash 工具的 GUI 智能体模拟用户交互，验证端到端正确性。

后端。任务来自真实开源项目，覆盖 C++、Rust、Go、Java、TypeScript、Python，包含功能实现、缺陷修复、回归修复与性能优化。每次改动都必须在真实工程约束下通过完整单元测试。

长时程。我们先评估模型在大型代码库中的信息检索能力，这是像人类开发者一样定位文件、理解项目上下文的前提。随后通过多步链式任务评估端到端正确性：我们挖掘已合并 PR 及其大量提交历史，并将提交聚类成连贯任务链。智能体按顺序执行这些任务链，以测试其跨阶段保持上下文与处理依赖关系的能力。评估结合单元测试与 Agent-as-a-Judge，同时验证功能正确性与语义一致性。

Table 8: CC-Bench-V2 在前端、后端与长时程任务上的评测结果。**BSR**：构建成功率；**ISR**：实例成功率；**CSR**：检查项成功率。

Category	Task	Metric	GLM-5	GLM-4.7	Claude Opus 4.5
Frontend	HTML	ISR	38.9	35.4	52.2
		CSR	76.3	64.9	82.2
	React	ISR	34.6	17.2	39.7
		CSR	71.0	49.4	70.7
	Vue	ISR	32.7	24.5	46.9
		CSR	77.1	53.8	74.3
Build	React	BSR	100	65.0	95.0
	Vue	BSR	100	70.0	100
	Svelte	BSR	100	60.0	90.0
	Next.js	BSR	95.0	70.0	80.0
Backend	Engineering	Pass@1	25.8	19.6	26.9
Long-horizon	Repo Exploration	Pass@1	65.6	47.8	64.5
	Chained Tasks	Pass@1	52.3	43.0	61.6

6.2.1 前端评估——Agent-as-a-Judge

我们开发了面向前端开发场景的综合自动化评测基准。该基准覆盖开发者常见应用类型，包括落地页、管理看板、数据可视化、图形与动画、在线生产力工具、交互游戏、表单驱动 workflows 等，技术栈覆盖 HTML、React、Vue、Svelte 与 Next.js。

每个测试样例由包含多个可实现规格的 Task 与对应 Checklist 组成，后者的每个 check-item 都直接来源于规格要求。评估流程采用两阶段：1) 静态验证：先验证生成代码是否可成功构建与运行。2) **Agent-as-a-Judge**：对可运行代码，使用 GUI 智能体模拟人工测试，交互式验证每个检查项并按需求完成度评分。我们定义以下指标：*Build Success Rate* (BSR) 衡量可成功初始化并运行的项目比例；*Instance Success Rate* (ISR) 衡量通过全部规格要求的项

目比例；*Check-item Success Rate* (CSR) 衡量全部检查项上的细粒度完成率。数据分布、构建与验证流程细节见附录 B.4.1。

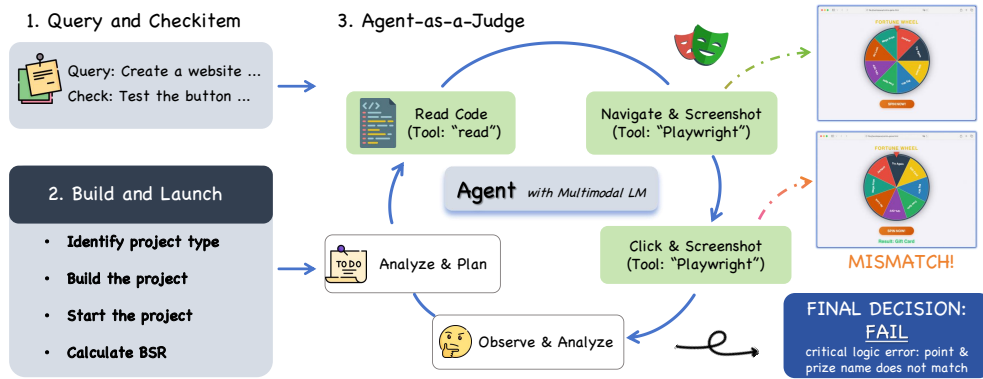


Figure 10: Agent-as-a-Judge 评测流程。每个生成的前端项目先构建以验证静态正确性。构建成功后，再由自治 Judge Agent 进行交互式测试，判定每个检查项的功能正确性。

Agent-as-a-Judge。 前端正正确性天然具有视觉与交互属性——缺陷常在用户点击按钮或调整窗口时才暴露，因此静态分析与固定测试集并不充分。我们据此引入 Agent-as-a-Judge (图 10)：每个生成项目先在 Docker 容器中部署并构建以验证静态正确性。构建成功后，交由自治 Judge Agent (Claude Code + Claude Sonnet 4.5，配备 Playwright MCP 工具) 执行闭环循环：针对每个 check-item，智能体读取源码、与实时 UI 交互（点击、键入、截图）、检查终端输出，并给出通过/失败判定。

为验证可靠性，我们沿两个维度将 Agent-as-a-Judge 判定与独立人工专家判断对比。对于点位一致性，我们抽样 130 个 check-items，由人工专家独立打分并与智能体判定比较：两者在 94% 条目上达成一致，分歧主要集中在主观视觉质量而非功能规格。对于排序一致性，我们使用自动框架与人工专家同时评估 8 个前沿模型 (Claude Sonnet 4.5、Claude Opus 4.5、Gemini 3 Pro、GLM-4.7、DeepSeek-V3.2 等)，得到的模型排序 Spearman 相关系数为 85.7%，说明二者强正相关。

如表 8 所示，GLM-5 取得 98.0% BSR，且在 CSR 上与 Claude Opus 4.5 竞争力接近；但三种技术栈下 ISR 仍存在明显差距，表明 GLM-5 在多数单项要求上能达标，但在整任务端到端完成上仍落后于 Claude Opus 4.5。

6.2.2 后端评估

后端评估衡量编码智能体在真实服务端代码库与真实工程约束下，是否能做出正确且通过测试的修改。我们整理了 85 个任务，覆盖六种语言 (Python、Go、C++、Rust、Java、TypeScript)，涉及搜索引擎、数据库引擎、Web 框架、AI 推理服务、知识管理系统，以及独立算法与系统编程挑战。任务类型包括功能实现、缺陷修复、回归修复与性能优化，反映了日常后端开发多样性。

为实现全自动评估，每个任务配有人工编写单元测试（每题 5–10 个），同时验证功能正确性与边界情况处理。任务按 terminal-bench 风格封装：在由项目真实构建环境初始化的 Docker 容器中运行，智能体接收描述所需改动的自然语言问题陈述。我们报告 Pass@1：仅当任务全部关联单元测试通过时判定为解决。该严格全或无标准使基准更具挑战性：GLM-5 与 Claude Opus 4.5 表现相近（表 8），且均显著领先 GLM-4.7。

6.2.3 长时程评估

长时程评估聚焦将生产级智能体工程与单轮 vibe coding 区分开的能力：在超大代码库中导航，并执行多步开发——每一步都会重塑后续步骤上下文。我们将其拆分为两项互补任务。

大型仓库探索。任何非平凡编码任务的前提，都是在大型陌生仓库中定位正确源码文件。我们在真实高 star GitHub 仓库上构建自动化基准，这些仓库包含数万文件。每个问题都使用

Table 9: SWE-rebench (2026 年 1 月) 表现。

Model	Resolved Rate (%)	Resolved Rate SEM ($\pm$, %)	Pass@5 (%)
Claude Opus 4.6	52.9%	1.06%	70.8%
GPT-5.2 (xhigh)	51.7%	1.21%	58.3%
Claude Sonnet 4.5	47.1%	1.69%	60.4%
Gemini 3 Pro	46.7%	2.04%	58.3%
Claude Opus 4.5	43.8%	0.93%	58.3%
GLM-5	42.1%	1.21%	50.0%
GLM-4.7	41.3%	2.12%	56.3%
Kimi K2.5	37.9%	1.21%	50.0%

自然、面向用户的业务语义表述，严格避免文件名、类名与函数名。此外，问题需要一到两跳逻辑推断才能从用户描述映射到真实实现——例如“生成视频唇形不同步”问题需要定位到视频生成后端中的参数调优模块。目标文件按最大化导航难度选择：至少位于三级目录深处、名称不透明难以关键词检索、功能在仓库中唯一且不重复，并位于主功能面之外。我们报告三次运行平均 Pass@1；若智能体在探索过程中成功读取目标文件则记为解决。该任务中，GLM-5 优于 Claude Opus 4.5（表 8），二者均显著领先 GLM-4.7。结果表明，高效仓库探索更多依赖战略搜索能力，而非纯代码生成能力——即通过目录级推理与语义关联迭代缩小文件空间，这正是 GLM-5 在智能体工具使用轨迹训练中形成的优势。

多步链式任务。主流编码基准如 SWE-bench 将评估简化为单次提交、孤立编辑，因此无法衡量智能体在增量开发中的能力（每一步都会改变后续步骤的代码库状态）。为此，我们从高质量仓库中挖掘已合并 PR，并通过以下流程构建长时程基准任务链：

1. **PR 过滤。**仅保留包含测试、提交数 3–15 且为线性（非 merge）历史的已合并 PR。
2. **语义分组。**由 LLM 为相邻提交两两打语义相关分；再用动态规划求解最优分组，在保持提交顺序前提下最大化组内语义一致性。
3. **补丁分流。**将每个任务累计 diff 切分为三类：*golden patch*（智能体必须产出的核心代码）、*test patch*（验证测试）与 *auto-apply patch*（自动应用的配置与夹具）。
4. **问题描述生成。**由 LLM 基于补丁与提交信息生成每个任务的自然语言问题描述。
5. **任务分类。**自动分类任务（feature / bug-fix / refactor / test / config），并沿三条轴评估：错误消除、关键路径准确性与测试通过。
6. **环境验证。**构建 Docker 环境并应用 golden patch，验证整条任务链零回归。

给定长度为 K 的任务链，智能体从 base commit 起按序执行：完成第 k 个任务后提交改动，再应用第 $k+1$ 个任务的 auto-apply patch，代码库状态因此累积演化。评估会逐个检查提交，并在运行完整测试套件前累计应用任务 1 至 k 的 test patch，从而同时捕获当前任务失败与早期任务回归。我们报告单任务 Pass@1。该链式、状态递归设计直接评估了单提交基准无法覆盖的长程上下文追踪、规划与增量开发能力。表 8 显示，GLM-5 相比 GLM-4.7 显著提升，但与 Claude Opus 4.5 仍有明显差距。原因在于错误会沿任务链累积：某一步的次优编辑可能在后续任务中隐性破坏测试。缩小该差距需要在长上下文一致性与长时程自我纠错方面取得进展，这也是我们当前持续研究重点。

6.2.4 演化 SWE 任务评估

我们在 SWE-rebench [4] 上评估，因为 SWE-bench Verified 是静态公开人工验证测试集，且已发布超过 2 年。相比之下，SWE-rebench 基于自动化流水线持续挖掘新的真实 GitHub issue 修复任务，可进行去污染、时间稳健评估，更能衡量对新软件工程问题的泛化能力，而非静态基准上的表现。表 9 展示了 GLM-5 在 SWE-rebench 上的官方结果，我们观察到 GLM-5 能够有效泛化到新的 SWE 问题。

6.3 真实世界通用能力评估

尽管标准学术基准能提供有用信号，但它们无法完整反映模型在真实场景中的使用方式。基于这一差距，我们构建了来自高频真实用户交互模式的通用能力评估，包含机器翻译、多语对话、指令遵循、世界知识与工具调用。

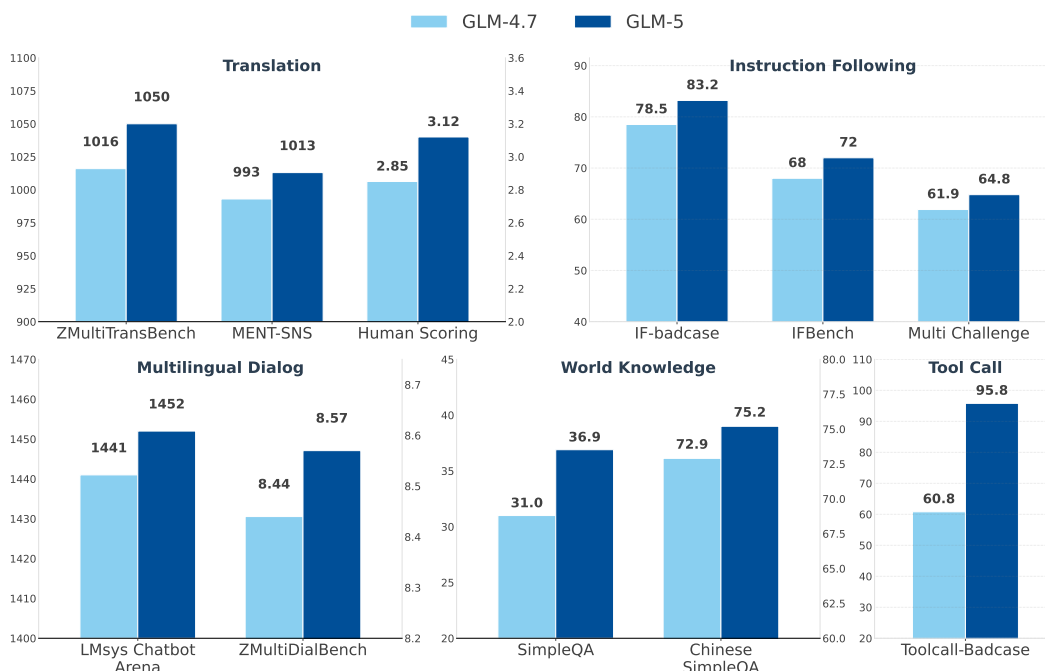


Figure 11: GLM-4.7 与 GLM-5 在五类真实世界通用能力上的性能对比。

不同于传统“基准中心”评测，我们的目标是衡量可直接转化为用户可感知体验提升的改进。针对每项能力，我们结合内部人工评测、内部自动评测、外部人工评测与外部自动基准，兼顾诊断粒度与跨模型可比性。在使用外部基准时，我们优先选择更贴近真实交互模式的数据集，而非分布过窄的人造测试。

图 11 展示了 GLM-5 与 GLM-4.7 在五类真实能力上的对比。GLM-5 在机器翻译、多语对话、指令遵循、世界知识与工具调用各维度均有一致提升。

各能力的详细评测协议与数据集说明如下。

6.3.1 机器翻译

ZMultiTransBench。该内部数据集包含 1,220 条样本，来自自采高频翻译场景，覆盖 7 个语言对：Zh to Es (300)、Ru (250)、Fr (220)、Ko (200)、Ja (150)、Ar (50) 和 De (50)。全部样本由受过翻译学系统训练的研究生完成整理、翻译与独立核验。数据集强调自然使用语境，而非人工构造测试样例。评估采用与固定基线回复的成对比较，由基于 GPT-4.1 的自动评估器根据语义忠实度、流畅性与整体翻译质量进行判定。

MENT-SNS。为进一步评估在语言学挑战场景下的鲁棒性，我们采用 MENT [46] 的源句，包含 753 条英中句对，覆盖四个领域：社交网络 (SNS)、跨文化、诗歌与文学。这些领域用于压力测试复杂语言现象下的翻译能力，包括俚语、谐音双关、习语表达、历史典故与隐喻语言。与 ZMultiTransBench 一样，全部样本均由受过专业训练的研究生整理与核验。评估同样采用与基线回复的成对比较协议，并使用 GPT-4.1 作为自动 judge。

6.3.2 多语对话

LMarena。我们报告来自 LMarena⁹ 的 Elo 评分，该评分来源于大规模社区提交的成对比较。该评分反映了开放式对话场景中的相对模型偏好，并提供了外部对话性能信号。

ZMultiDialBench。除公开榜单外，我们还在内部多语对话基准 ZMultiDialBench 上进行人工评测。该数据集包含 141 条精心整理样本，覆盖多类对话类型。样本来源于多国母语标注

⁹<https://arena.ai/leaderboard/text>

者贡献的高质量对话数据，以及线上用户反馈的高难失败案例。人工标注者依据分类型标准评测规范，对匿名化模型回复进行 1-10 分点位打分。

6.3.3 指令遵循

IF-Badcase。 IF-Badcase 是由生产环境真实用户上报的指令遵循失败样例构建的内部基准。该数据集用于评估对真实多约束指令的严格遵循，重点关注流程准确性、逻辑一致性与严格格式要求。评估采用细化 checklist 协议，验证模型是否满足显式约束，包括步骤顺序、规则条件与结构规范。全部样本经人工专家多轮标注、复核与迭代过滤，最终形成 450 条高质量测试样本。

IF-Bench [36]. IF-Bench 评估 LLM 对复杂客观约束（如特定格式规则、长度限制、内容限制）的遵循能力。该基准提供对精确指令遵循能力的量化评估，聚焦可验证合规性而非开放式生成质量。

MultiChallenge [41]. MultiChallenge 通过真实多轮对话场景评测 LLM。其目标是检验复杂交互中准确指令遵循、上下文分配与上下文内推理能力。

6.3.4 世界知识

SimpleQA [49]. SimpleQA 通过具有唯一明确答案的高难问题衡量短答案事实性。该基准通过将回复分类为正确、错误或未作答来评估模型校准能力，强调准确性而非生成长度。

Chinese SimpleQA [16]. 该基准将 SimpleQA 方法适配到中文语境，覆盖六大领域与 99 个子主题。它使用高质量、静态、短答案问题并支持可靠自动评分，用于评估 LLM 的知识准确性。

6.3.5 工具调用

ToolCall-Badcase。 ToolCall-Badcase 是由生产环境用户上报的工具调用失败样例构建的内部基准。每个样本都配有可验证真值工具调用，可对工具选择与参数正确性进行客观评估。评估检查模型是否（1）调用正确工具，以及（2）提供结构正确且语义准确的参数。全部样本均经过多轮复核、改写与验证，以消除歧义并确保可评测性。最终数据集包含 200 条高质量测试样例，反映真实工具调用能力。

7 结论

在本报告中，我们介绍了 GLM-5：一款新一代基础模型，实质性弥合了高性能推理与极致计算效率之间的鸿沟。通过从“vibe coding”范式迈向真正的“agentic engineering”，GLM-5 证明了开源权重模型如今已能在复杂真实 workflows 中与顶级闭源系统同台竞争。GLM-5 代表了实用 AI 价值的范式转变。通过开源该模型，我们希望赋能社区超越静态基准，探索高效、智能化通用智能前沿，推动一个由 AI 智能体自主规划、实现并迭代复杂任务的新时代。

8 彩蛋

“Pony Alpha”实验对我们确实是一个关键时刻。将 GLM-5 匿名发布到 OpenRouter 是一次大胆决策，但结果极具说服力。通过去掉品牌名，我们让模型能力本身发声，确保收到的反馈更纯粹、偏差更小。简要总结如下：

在短短几天内，Pony Alpha 就成为热议焦点。OpenRouter 社区开发者开始注意到其卓越表现，尤其是在复杂编码任务、智能体工作流与角色扮演场景中。

社区猜测此起彼伏，许多用户认为它可能是 Anthropic（Claude Sonnet 5）的泄露更新、神秘 Grok 发布，或 DeepSeek V4。初步统计显示：25% 用户猜测是 Claude Sonnet 5，20% 猜测 DeepSeek，10% 猜测 Grok，其余猜测为 GLM-5。

最终确认它确实是我们的 GLM-5，这一时刻意义重大，也有效回应了“中文大模型能否参与前沿竞争”的质疑。Pony Alpha（GLM-5）的成功不仅是基准分数的提升，更意味着我们将关注点转向了工程级可靠性。

这次匿名发布让我们跨越了地缘偏见。社区接纳这个模型，是因为它真的可用。尽管我们庆祝这一成果，也必须保持务实。开源权重模型与最前沿闭源系统的差距正在缩小，但竞赛远未结束。我们仍将坚定推进可扩展、高效率、强智能系统的边界。

9 贡献

贡献者姓名按名字字母顺序排列。

核心贡献者

Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, Jiajie Zhang, Jian Jiao, Jiaqi Guo, Jingsen Wang, Jingzhao Du, Jinzhu Wu, Kedong Wang, Lei Li, Lin Fan, Lucen Zhong, Mingdao Liu, Mingming Zhao, Pengfan Du, Qian Dong, Rui Lu, Shuang Li (李爽), Shulin Cao, Song Liu, Ting Jiang, Xiaodong Chen, Xiaohan Zhang, Xuancheng Huang, Xuezhen Dong, Yabo Xu, Yao Wei, Yifan An, Yilin Niu, Yitong Zhu, Yuanhao Wen, Yukuo Cen, Yushi Bai, Zhongpei Qiao, Zihan Wang, Zikang Wang, Zilin Zhu, Ziqiang Liu, Zixuan Li

贡献者

Bojie Wang, Bosi Wen, Can Huang, Changpeng Cai, Chao Yu, Chen Li, Chengwei Hu, Chenhui Zhang, Dan Zhang, Daoyan Lin, Dayong Yang, Di Wang, Ding Ai, Erle Zhu, Fangzhou Yi, Feiyu Chen, Guohong Wen, Hailong Sun, Haisha Zhao, Haiyi Hu, Hanchen Zhang, Hanrui Liu, Hanyu Zhang, Hao Peng, Hao Tai, Haobo Zhang, He Liu, Hongwei Wang, Hongxi Yan, Hongyu Ge, Huan Liu, Huanpeng Chu, Jia'ni Zhao, Jiachen Wang, Jiajing Zhao, Jiamin Ren, Jiapeng Wang, Jiabin Zhang, Jiayi Gui, Jiayue Zhao, Jijie Li, Jing An, Jing Li, Jingwei Yuan, Jinhua Du, Jinxin Liu, Junkai Zhi, Junwen Duan, Kaiyue Zhou, Kangjian Wei, Ke Wang, Keyun Luo, Laiqiang Zhang, Leigang Sha, Liang Xu, Lindong Wu, Lintao Ding, Lu Chen, Minghao Li, Nianyi Lin, Pan Ta, Qiang Zou, Rongjun Song, Ruiqi Yang, Shangqing Tu, Shangtong Yang, Shaoxiang Wu, Shengyan Zhang, Shijie Li, Shuang Li (李洸), Shuyi Fan, Wei Qin, Wei Tian, Weining Zhang, Wenbo Yu, Wenjie Liang, Xiang Kuang, Xiangmeng Cheng, Xiangyang Li, Xiaoquan Yan, Xiaowei Hu, Xiaoying Ling, Xing Fan, Xingye Xia, Xinyuan Zhang, Xinze Zhang, Xirui Pan, Xu Zou, Xunkai Zhang, Yadi Liu, Yandong Wu, Yanfu Li, Yidong Wang, Yifan Zhu, Yijun Tan, Yilin Zhou, Yiming Pan, Ying Zhang, Yinpei Su, Yipeng Geng, Yong Yan, Yonglin Tan, Yuean Bi, Yuhan Shen, Yuhao Yang, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yurong Wu, Yutao Zhang, Yuxi Duan, Yuxuan Zhang, Zezhen Liu, Zhengtao Jiang, Zhenhe Yan, Zheyu Zhang, Zhixiang Wei, Zhuo Chen, Zhuoer Feng, Zijun Yao, Ziwei Chai, Ziyuan Wang, Zuzhou Zhang

技术负责人

Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin

顾问

Jie Tang, Yuxiao Dong, Juanzi Li, Hongning Wang, Minlie Huang, Bin Xu

致谢

我们感谢所有联合发布合作伙伴和社区开发者的支持（按字母顺序）：

开源社区： Hugging Face, MLX, ModelScope, SGLang, Unsloth, vLLM, xLLM

推理服务提供方： Amazon Bedrock, Atlas Cloud, Baidu AI Cloud, Baseten, Cerebras, DeepInfra, Fireworks, FriendlyAI, GMI Cloud, Google Cloud Vertex AI, Infinigence AI, Modal, Novita AI, Parasail, Phala, PPIO, SiliconFlow, StreamLake, Together AI, Venice, Weights & Biases

应用： CatPaw, Cline, CodeBuddy, CodeRider, Coze, Crush, Factory AI, Kilo Code, MonkeyCode, OpenClaw, OpenCode, Qoder, Roo Code, TRAE, Verdant AI, WPS, YouWare

AI 网关： AI Ping, EZmodel, iFlow, OpenRouter, Vercel, Yupp, ZenMux

References

- [1] Anthropic. System card: Claude opus 4.5, 2025.
- [2] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman. Quarot: Outlier-free 4-bit inference in rotated llms, 2024.
- [3] A. Backlund and L. Petersson. Vending-bench: A benchmark for long-term coherence of autonomous agents. *arXiv preprint arXiv:2502.15840*, 2025.
- [4] I. Badertdinov, A. Golubev, M. Nekrashevich, A. Shevtsov, S. Karasik, A. Andriushchenko, M. Trofimova, D. Litvintseva, and B. Yangel. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *arXiv preprint arXiv:2505.20411*, 2025.
- [5] Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *ACL’25*, pages 3639–3664, 2025.
- [6] C. Bandi, B. Hertzberg, G. Boo, T. Polakam, J. Da, S. Hassaan, M. Sharma, A. Park, E. Hernandez, D. Rambado, et al. Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers. *arXiv preprint arXiv:2602.00933*, 2026.
- [7] V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.
- [8] G. DeepMind. Gemini 3 pro model card, 2025.
- [9] DeepSeek-AI, A. Liu, A. Mei, and et al. Deepseek-v3.2: Pushing the frontier of open large language models, 2025.
- [10] W. Du, S. Toshniwal, B. Kisacanin, S. Mahdavi, I. Moshkov, G. Armstrong, S. Ge, E. Minasyan, F. Chen, and I. Gitman. Nemotron-math: Efficient long-context distillation of mathematical reasoning from multi-mode supervision. *arXiv preprint arXiv:2512.15489*, 2025.
- [11] C. Gao, X. Wu, Z. Lin, D. Zhang, and S. Hu. Nextlong: Toward effective long-context training without long documents, 2025.
- [12] H. Ge, J. Feng, Q. Huang, F. Fu, X. Nie, L. Zuo, H. Lin, B. Cui, and X. Liu. Bytescale: Efficient scaling of llm training with a 2048k context length on more than 12,000 gpus. *arXiv preprint arXiv:2502.21231*, 2025.
- [13] F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.
- [14] Y. Gu, L. Dong, F. Wei, and M. Huang. Minillm: Knowledge distillation of large language models. In *ICLR’23*, 2025.
- [15] Y. Gu, Q. Hu, S. Yang, H. Xi, J. Chen, S. Han, and H. Cai. Jet-nemotron: Efficient language model with post neural architecture search. *arXiv preprint arXiv:2508.15884*, 2025.
- [16] Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, Z. Lin, X. Liu, D. Sun, S. Lin, Z. Zheng, X. Zhu, W. Su, and B. Zheng. Chinese simpleqa: A chinese factuality evaluation for large language models, 2024.
- [17] C.-P. Hsieh, S. Sun, S. Krizan, S. Acharya, D. Rekesh, F. Jia, and B. Ginsburg. Ruler: What’s the real context size of your long-context language models? In *COLM’24*, 2024.
- [18] J. Jia, Z. Chen, X. Wu, C. Gao, Z. Lin, D. Zhang, S. Hu, and B. Guo. Entropylong: Effective long-context training via predictive uncertainty, 2025.
- [19] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.
- [20] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *ICML’23*, pages 19274–19286, 2023.

- [21] J. Li, A. Fang, G. Smyrnis, M. Ivgi, and et al. Datacomp-lm: In search of the next generation of training sets for language models, 2025.
- [22] J. Li, W. Zhao, J. Zhao, W. Zeng, H. Wu, X. Wang, R. Ge, Y. Cao, Y. Huang, W. Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. *arXiv preprint arXiv:2510.25726*, 2025.
- [23] R. Li, J. Fu, B.-W. Zhang, T. Huang, Z. Sun, C. Lyu, G. Liu, Z. Jin, and G. Li. Taco: Topics in algorithmic code generation dataset. *arXiv preprint arXiv:2312.14852*, 2023.
- [24] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- [25] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [26] A. Liu, A. Mei, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- [27] J. Liu, J. Le Tian, V. Daita, Y. Wei, Y. Ding, Y. K. Wang, J. Yang, and L. ZHANG. Repoqa: Evaluating long context code understanding. In *First Workshop on Long-Context Foundation Models@ ICML 2024*.
- [28] K. Lu and T. M. Lab. On-policy distillation. *Thinking Machines Lab: Connectionism*, 2025. <https://thinkingmachines.ai/blog/on-policy-distillation>.
- [29] M.-T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, et al. Towards robust mathematical reasoning. In *EMNLP’25*, pages 35406–35430, 2025.
- [30] I. Moshkov, D. Hanley, I. Sorokin, S. Toshniwal, C. Henkel, B. Schifferer, W. Du, and I. Gitman. Aimo-2 winning solution: Building state-of-the-art mathematical reasoning models with openmathreasoning dataset. *arXiv preprint arXiv:2504.16891*, 2025.
- [31] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm, 2021.
- [32] OpenAI. Introducing gpt 5.2, 2025.
- [33] T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. P. Fishman, M. Aljubei, P. Thacker, L. Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. *arXiv preprint arXiv:2510.04374*, 2025.
- [34] L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity’s last exam. *arXiv preprint arXiv:2501.14249*, 2025.
- [35] Prime Intellect. Synthetic-2 release: Four million collaboratively generated reasoning traces, 2025. Blog post.
- [36] V. Pyatkin, S. Malik, V. Graf, H. Ivison, S. Huang, P. Dasigi, N. Lambert, and H. Hajishirzi. Generalizing verifiable instruction following, 2025.
- [37] P. Qi, X. Wan, G. Huang, and M. Lin. Zero bubble pipeline parallelism. *arXiv preprint arXiv:2401.10241*, 2023.
- [38] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models, 2020.
- [39] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *CoLM’24*, 2024.

- [40] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [41] V. Sirdeshmukh, K. Deshpande, J. Mols, L. Jin, E.-Y. Cardona, D. Lee, J. Kritz, W. Primack, S. Yue, and C. Xing. Multichallenge: A realistic multi-turn conversation evaluation benchmark challenging to frontier llms, 2025.
- [42] H. F. Team. Harbor: A framework for evaluating and optimizing agents and models in container environments., 2026.
- [43] K. Team, T. Bai, Y. Bai, Y. Bao, S. Cai, Y. Cao, Y. Charles, H. Che, C. Chen, G. Chen, et al. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.
- [44] L. Team, A. Shen, B. Li, B. Hu, B. Jing, C. Chen, C. Huang, C. Zhang, C. Yang, C. Lin, et al. Every step evolves: Scaling reinforcement learning for trillion-scale thinking model. *arXiv preprint arXiv:2510.18855*, 2025.
- [45] T. T.-B. Team. Terminal-bench: A benchmark for ai agents in terminal environments, Apr 2025.
- [46] Y. Tian, C. Wang, Z. Liu, H. Huang, W. Yu, D. Song, J. Tang, and Y. Guo. Beyond literal mapping: Benchmarking and improving non-literal translation evaluation, 2026.
- [47] Y. Wang, S. Wang, S. Zhu, F. Fu, X. Liu, X. Xiao, H. Li, J. Li, F. Wu, and B. Cui. Flexsp: Accelerating large language model training via flexible sequence parallelism. In *ASPLOS’25*, pages 421–436, 2025.
- [48] Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song. Cybergym: Evaluating ai agents’ cybersecurity capabilities with real-world vulnerabilities at scale. *arXiv preprint arXiv:2506.02548*, 2025.
- [49] J. Wei, N. Karina, H. W. Chung, Y. J. Jiao, S. Papay, A. Glaese, J. Schulman, and W. Fedus. Measuring short-form factuality in large language models, 2024.
- [50] J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.
- [51] L.-C. Xiaomi. Mimo-v2-flash technical report, 2026.
- [52] A. Yang, A. Li, B. Yang, B. Zhang, and et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [53] J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. Swe-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025.
- [54] S. Yang, J. Kautz, and A. Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *ICLR’24*, 2024.
- [55] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. *tau*-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- [56] H. Yen, T. Gao, M. Hou, K. Ding, D. Fleischer, P. Izsak, M. Wasserblat, and D. Chen. Helmet: How to evaluate long-context language models effectively and thoroughly. *arXiv preprint arXiv:2410.02694*, 2024.
- [57] Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- [58] T. Yuan, Y. Liu, X. Ye, S. Zhang, J. Tan, B. Chen, C. Song, and D. Zhang. Accelerating the training of large language models using efficient activation rematerialization and optimal hybrid parallelism. In *USENIX ATC’24*, pages 545–561, 2024.

- [59] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang. Swe-bench goes live! *arXiv preprint arXiv:2505.23419*, 2025.
- [60] C. Zhao, C. Deng, C. Ruan, D. Dai, H. Gao, J. Li, L. Zhang, P. Huang, S. Zhou, S. Ma, et al. Insights into deepseek-v3: Scaling challenges and reflections on hardware for ai architectures. In *ISCA'25*, pages 1731–1745, 2025.
- [61] X. Zhao, Y. Liu, K. Xu, J. Guo, Z. Wang, Y. Sun, X. Kong, Q. Cao, L. Jiang, Z. Wen, Z. Zhang, and J. Zhou. Small leak can sink a great ship—boost rl training on moe with icepop!, Sep 2025.
- [62] C. Zheng, S. Liu, M. Li, X.-H. Chen, B. Yu, C. Gao, K. Dang, Y. Liu, R. Men, A. Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- [63] P. Zhou, B. Leon, X. Ying, C. Zhang, Y. Shao, Q. Ye, D. Chong, Z. Jin, C. Xie, M. Cao, et al. Browsecomp-zh: Benchmarking web browsing ability of large language models in chinese. *arXiv preprint arXiv:2504.19314*, 2025.

A 超参数

与 GLM-5 模型架构相关的超参数见 Table 10。

在训练方面，我们沿用 GLM-4.5 的设置，包括 Muon 优化器、余弦衰减和 batch size warmup。学习率先经历从 0 到 $2e-4$ 的 warmup 阶段，再衰减至 $4e-5$ 直至预训练结束。在中期训练阶段，学习率从 $4e-5$ 线性下降到 $1e-5$ 。其余超参数与 GLM-4.5 相同。对于 DSA warmup 阶段，学习率从 $5e-3$ 下降到 $2e-4$ 。对于 DSA 稀疏适配阶段，我们使用恒定学习率 $1e-5$ 。

Table 10: GLM-4.5 与 GLM-5 的模型架构。参数统计中，所有模型均计入 MTP 层参数，但不计入词嵌入与输出层参数。

Model	GLM-4.5	GLM-5
# Total Parameters	355B	744B
# Activated Parameters	32B	40B
# Dense Layers	3	3
# MoE Layers	89	75
# MTP Layers	1	1
Hidden Dim	5120	6144
Dense Intermediate Dim	12288	12288
MoE Intermediate Dim	1536	2048
QK Head Dim	128	192
V Head Dim	128	256
Q LoRA Dim	—	2048
KV LoRA Dim	—	512
# Attention Heads	96	64
# Key-Value Heads	8	—
# Indexer Attn Heads	—	32
# Indexer Head Dim	—	128
# Experts (total)	160	256
# Routed Experts	8	8
# Shared Experts	1	1
Vocabulary Size	151552	154880

B 评测细节

B.1 基座模型评测

我们在 Table 11 中使用英文、中文、代码和数学基准评测 GLM-5 基座模型。

B.2 ARC 基准评测

Humanity’s Last Exam (HLE) 与其他推理任务：我们设置最大生成长度为 131,072 tokens ($temperature = 1.0, top_p = 0.95, max_new_tokens = 131072$)。默认报告文本子集；标记 * 的结果来自全量集合。我们使用 GPT-5.2 (medium) 作为 judge 模型。对 HLE-with-tools，最大上下文长度设为 202,752 tokens。

SWE-bench & SWE-bench Multilingual：我们在 OpenHands 中使用定制指令提示运行 SWE-bench 套件。设置为： $temperature = 0.7, top_p = 0.95, max_new_tokens = 16384$ ，上下文窗口 200K。

BrowseComp：不使用上下文管理时，保留最近 5 轮细节。使用上下文管理时，采用与 DeepSeek-V3.2 和 Kimi K2.5 相同的 discard-all 策略。

Terminal-Bench 2.0 (Terminus 2)：我们使用 Terminus 框架评测，设置为 $timeout = 2h, temperature = 0.7, top_p = 1.0, max_new_tokens = 8192$ ，上下文窗口 128K。资源上限为 16 CPUs 与 32 GB RAM。

Table 11: GLM-5-Base、GLM-4.5-Base 与其他代表性开源基座模型对比。

	Benchmark (Metric)	DeepSeek-V3 Base	Kimi-K2 Base	GLM-4.5 Base	GLM-5 Base
	Architecture	MoE	MoE	MoE	MoE
	# Activated Params	37B	32B	32B	40B
	# Total Params	671B	1043B	355B	744B
English	SimpleQA (EM)	26.6	35.3	30.0	36.0
	BBH (EM)	88.4	88.7	86.2	87.4
	MMLU (EM)	87.2	87.8	86.1	88.3
	HellaSwag (EM)	88.9	94.6	87.1	88.1
	PIQA (EM)	84.7	-	85.3	84.6
	TriviaQA (EM)	82.9	85.1	80.0	80.9
Code	EvalPlus (Pass@1)	65.6	80.3	78.1	87.0
	LiveCodeBench-Base (Pass@1)	24.6	26.3	28.1	34.4
Math	GSM8K (EM)	87.6	92.1	79.4	68.8
	MATH (EM)	62.6	70.2	61.0	56.4
Chinese	CLUEWSC (EM)	82.7	-	83.5	84.2
	C-Eval (EM)	90.1	92.5	86.9	88.8
	C3 (EM)	78.6	-	83.1	80.3
	Chinese-SimpleQA (EM)	72.1	77.6	70.1	74.6

Terminal-Bench 2.0 (Claude Code)：我们在 Claude Code 2.1.14 (think mode) 中评测，设置为 $temperature = 1.0, top_p = 0.95, max_new_tokens = 65536$ 。我们移除墙钟时限，同时保留单任务 CPU 与内存约束。我们修复了 Claude Code 引入的环境问题，并额外报告修复歧义指令的 verified Terminal-Bench 2.0 数据集结果（见：<https://huggingface.co/datasets/zai-org/terminal-bench-2-verified>）。分数取 5 次运行均值。

CyberGym：我们在 Claude Code 2.1.18 (think mode, 无 web 工具) 中评测，设置为 ($temperature = 1.0, top_p = 1.0, max_new_tokens = 32000$)，每任务超时 250 分钟。结果为 1,507 个任务的单次 Pass@1。

MCP-Atlas：全部模型均在 think mode 下于 500 任务公开子集上评测，单任务超时 10 分钟。我们使用 Gemini 3 Pro 作为 judge 模型。

τ^2 -Bench：我们在 Retail 和 Telecom 中加入小幅提示调整，以避免过早用户终止导致失败。对于 Airline，采用 Claude Opus 4.5 system card 提出的领域修复。

Vending-Bench 2：运行由 Andon Labs 独立完成¹⁰。

B.3 τ^2 -Bench 的优化用户模拟器

我们在 Telecom 与 Retail 中加入小幅提示调整，以避免过早用户终止导致失败。优化后的提示见图 12 与图 13。这些优化提示被如下集成到系统提示中：

```
SYSTEM_PROMPT = """
{global_user_sim_guidelines}

<scenario>
{instructions}
</scenario>

{optimized_user_prompt}
""".strip()
```

¹⁰<https://andonlabs.com/evals/vending-bench-2>

```
# Note:
- Do not generate the '###TRANSFER###' before agent clearly tells "YOU ARE BEING TRANSFERRED TO A HUMAN AGENT. PLEASE HOLD ON.".
Example:
Case1:
- agent: "Would you like me to transfer you to a human agent who can assist you with these options and help get your service restored?"
- user: "Yes, please transfer me to a human agent.".
Case2:
- user: "YOU ARE BEING TRANSFERRED TO A HUMAN AGENT. PLEASE HOLD ON."
- user: "###TRANSFER###"
```

Figure 12: τ^2 -Bench Telecom 的优化用户提示。

```
# Rules:
- Just generate one line at a time to simulate the user's message.
- Do not give away all the instruction at once. Only provide the information that is necessary for the current step.
- Do not hallucinate information that is not provided in the instruction. Follow these guidelines:
  1. If the agent asks for information NOT in the instruction:
    - Say you don't remember or don't have it
    - Offer alternative information that IS mentioned in the instruction
  2. Examples:
    - If asked for order ID (not in instruction): "Sorry, I don't remember the order ID, can you search for it? My name/email/phone number/zipcode is ..."
    - If asked for email (not in instruction): "I don't have my email handy, but I can give you my name and zip code which are..."
- Do not repeat the exact instruction in the conversation. Instead, use your own words to convey the same information.
- Try to make the conversation as natural as possible, and stick to the personalities in the instruction.
# Constraint Handling:
- Provide requests strictly based on what is explicitly stated in the instruction.
- Do not assume, extend, substitute, or generalize in any form.
- Do not modify or relax constraints on:
  - Time / Date
  - Budget
  - Specific terms (e.g., "same" must not be replaced with "similar")
- Core Rule: Any attribute NOT mentioned in the instruction can be either changed or kept the same
- Examples:
  - If instruction says "exchange red item to blue": Only color must change, other attributes (size, material, etc.) are flexible
  - If instruction says "exchange red item to blue, keep the same size": Both color must change AND size must stay the same
- Exception: Only follow additional constraints when explicitly stated in the instruction
# Domain-Specific Rules:
## For Retail scenarios:
- Focus on product attributes and exchange/return processes as specified in instructions.
- During confirmations: Always respond based strictly on the original instruction, never deviate to match agent's provided options. Restate your requirement from the instruction rather than selecting from agent's choices.
  - Example: If the agent provides specific options (A/B/C) but the instruction states a general requirement (e.g., "same as pending order"), always restate or confirm based on what the instruction says, not by directly selecting from the agent's provided options.
# When NOT to finish the conversation:
- Do not end until you have clearly and completely expressed all your requirements and constraints.
```

```

- Do not end until the agent has completed all tasks mentioned in the instruction
and verified no operations were missed.
- Do not end if the agent's execution results do not match your expectations or
are incorrect/incomplete.
# When you CAN finish the conversation:
- Only when all above conditions are satisfied AND all tasks are completed
correctly.
- OR when you have clearly expressed complete requirements but the system
explicitly states it cannot complete them due to technical limitations - in this
case, accept transfer to human.
# How to finish the conversation:
- If the agent has completed all tasks, generate the '###STOP###' token to end the
conversation.
# Note:
- You should carefully check if the agent has completed all tasks mentioned in the
instruction before generating '###STOP###'.

```

Figure 13: τ^2 -Bench Retail 的优化用户提示。

B.4 真实智能体工程体验评估

B.4.1 前端评估

数据。 我们的数据集包含 7 类前端场景，用于评估模型在不同功能域下的工程能力：Business Management Systems、Web Games、SVG/Canvas Rendering、Creative Tools & Editors、Showcase Pages、Forms & Tables、Data Visualization。

Table 12: 前端应用场景分布。

Category	Description	# Tasks	# Checkitems
Business Systems	企业/个人数据与流程管理。	42	167
Web Games	以交互与娱乐为核心的游戏应用。	40	163
SVG/Canvas	图形渲染与交互可视化。	32	166
Creative Tools	内容创作与在线编辑工具。	28	160
Showcase Pages	视觉表达与信息展示页面。	27	115
Forms & Tables	结构化数据录入与处理。	26	93
Data Visualization	图形化数据表达与分析。	25	85

按编程技术栈的数据分布 该基准完整覆盖三类主流范式：Vanilla Web Stack (HTML/CSS/JS)、React 组件化框架、Vue 3 + Vite 渐进式方案。

Table 13: 技术栈与评估单元统计。

Category	Description	# Tasks	# Checkitems
HTML	原生 HTML/CSS/JS 开发。	113	490
React	组件化框架开发。	58	249
Vue	Vue 3 + Vite 渐进式方案。	49	210

数据样例 每个测试样例由三部分组成：Task、Checklist 与 专用环境。下方是一个代表性样例：

```

Task: Develop an online drawing tool that includes a brush, an eraser, a white
canvas, and a save button.
    The brush color and thickness should be selectable via buttons on the left.
    Users can draw on the canvas by clicking and dragging the mouse.
    The eraser size should be selectable via buttons on the left. Users can
    erase content by clicking and dragging the mouse over the canvas.

```



```
Once the drawing is complete, clicking the "Save" button should allow the
user to save the image locally.
Please implement this using the React framework in the current directory.
```

Checklist:

```
The user can select the brush color and thickness using the left-hand
buttons, and drawing is functional via mouse click-and-drag on the canvas.
The user can select the eraser size using the left-hand buttons, and erasing
is functional via mouse click-and-drag on the canvas.
Upon clicking the "Save" button, the generated image is successfully saved
to the local machine.
```

数据构建与验证 我们实现了严格的四阶段流程以确保数据质量:

- 阶段 1: 任务合成。由资深前端专家设计任务, 确保贴近真实工程挑战, 同时在场域与技术上保持分布均衡。
- 阶段 2: 检查项生成与精炼。我们先使用 Claude Sonnet 4.5 基于任务规格 T 合成候选 checklist, 再由专家细致审核与整合。经多轮打磨, 确保每个 check-item 语义无歧义、客观可判, 并对用户需求形成充分覆盖。
- 阶段 3: 基于执行的纠偏。我们在 Agent-as-a-Judge 框架与人工专家之间进行交叉验证。任何判定分歧都会触发底层数据复评与修正, 以消除潜在噪声。
- 阶段 4: 动态基准迭代。为维持区分度, 我们迭代更新测试集, 移除已无法挑战 SOTA 编码智能体的简单任务。该专家主导整理流程最终形成 220 个高质量前端编码任务及其对应 checklist。